

Руководство по разработке сценариев для Russian Railway Simulator (RRS)

[Введение](#)

[1. Инструментальные средства для разработки сценария](#)

[2. Структура сценария. Простейшие функции](#)

[3. Более сложные команды сценариев. Позиционные триггеры](#)

[4. Продвинутое приемы разработки сценариев](#)

[5. График движения поезда](#)

[Приложение 1. Перечень системных функций](#)

[Приложение 2. Перечень action-функций для подстановки в триггеры](#)

[Приложение 3. Стандартные позиционные триггеры](#)

[Приложение 4. Стандартные структуры данных сценариев](#)

[Приложение 5. Формат графика движения поезда](#)

Введение

Система сценариев симулятора предназначена для описания событий, происходящих во время игры. Она обеспечивает расстановку поездов и управление поездными и маневровыми маршрутами их передвижения, позволяя имитировать для одного или нескольких игроков деятельность участка железной дороги.

Сценарий RRS - это программа на скриптовом языке [Lua](#). Тех, кто уже испугался того, что разработчики снова заставляют вас программировать, поспешу успокоить - для большинства типовых действий система сценариев RRS предоставляет простые и понятные команды, которые будут описаны в данном руководстве.

Вместе с тем, использование полноценного языка программирования позволит продвинутому сценаристу реализовать сколь угодно сложную логику игры. Для этого придется погрузиться в [нюансы и особенности языка Lua](#). Полноценный интерпретатор этого языка уже встроен в игру.

1. Инструментальные средства для разработки сценария

Для разработки сценариев необходимы следующие программные инструменты:

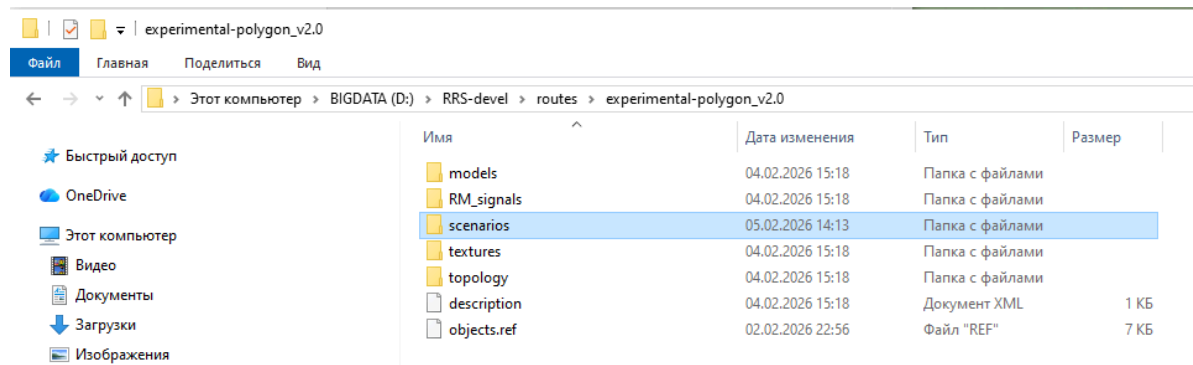
1. Текстовый редактор. Подойдет даже "Блокнот", однако, разработчики **крайне не рекомендуют использовать редактора "Блокнот"**! Лучше скачать и установить что-то более функциональное, гарантирующее поддержку сохранения файлов в кодировке UTF-8. Примеры: [Notepad++](#), [Sublime Text](#).
2. Приложение "Карта маршрута", входящее в комплект симулятора (bin/route-map.exe). Необходимо для получения информации о путевой инфраструктуре маршрута, имен траекторий, стрелок и светофоров.

ВНИМАНИЕ!!! Пользователям, использующим симулятор в операционной системе Windows, следует убедиться, что выбранный вами текстовый редактор сохраняет файлы сценария в кодировке UTF-8 (Юникод)! В противном случае разработчики не гарантируют корректного исполнения сценариев симулятором.

2. Структура сценария. Простейшие функции

2.1. Простейший сценарий. Установка игрового времени и расположения поезда игрока

Любой сценарий пишется для конкретного маршрута в игре. Поэтому, естественным образом, все сценарии для конкретного маршрута должны располагаться в папке `scenarios`, расположенной в папке с маршрутом. Например, в состав игры входит маршрут "Испытательный полигон". Папка `scenarios` уже есть там

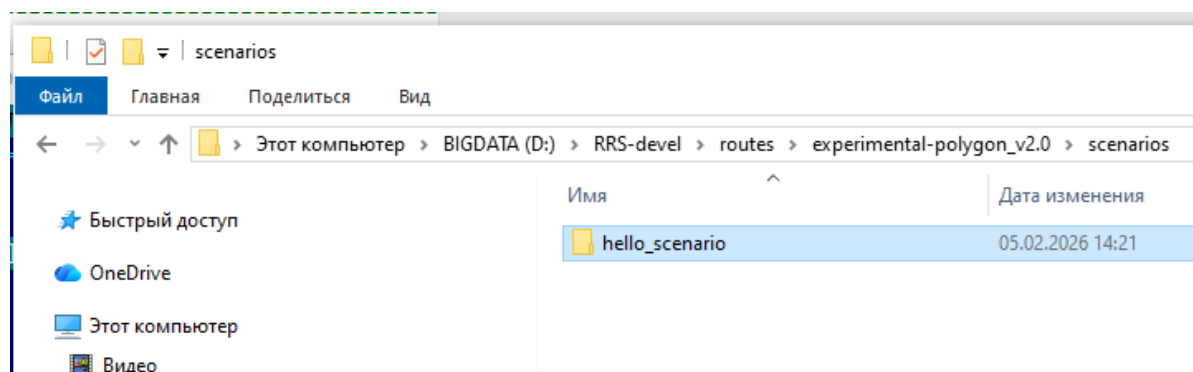


Если в том маршруте, для которого вы ведете разработку сценария, такой папки нет - создайте её!

В данной папке располагаются папки сценариев. Сценарий - это папка, содержащая файлы скриптов, файл описания, а в дальнейшем и звуковые файлы для озвучивания сценариев также будут располагаться там.

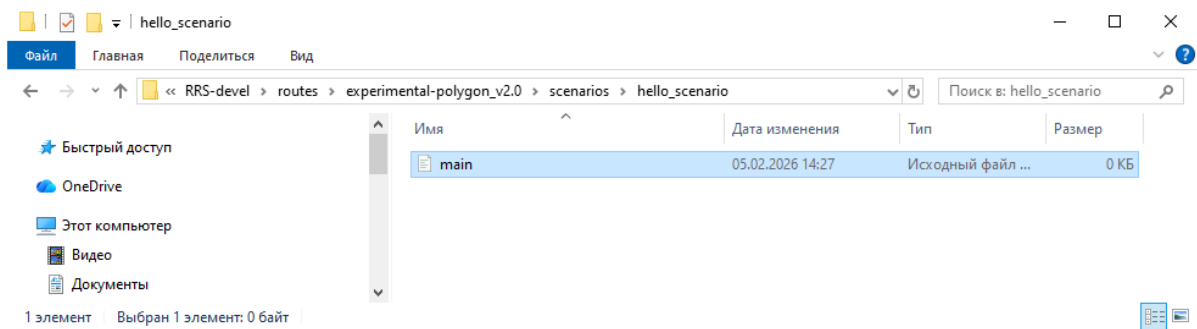
Давайте создадим свой первый сценарий в RRS! Для примера будем использовать стандартный маршрут "Испытательный полигон".

Создадим в папке `scenarios` папку с именем сценария. Дадим ей имя `hello_scenario`



ВНИМАНИЕ!!! Избегайте использования в именах папок и файлов сценариев национальных алфавитов и пробельных символов! Несмотря на то, что такие имена поддерживаются как ОС Windows, так и, по-видимому, движком Lua симулятора, не исключены проблемы при поиске и исполнении сценариев игрой! Только латиница, цифры, тире и подчеркивания.

Внутри этой папки создадим файл с именем `main.lua`



ВНИМАНИЕ!!! Файл с именем main.lua обязан присутствовать в сценарии! Это так называемый корневой или стартовый скрипт, с которого начинается исполнение сценария. Игра ожидает наличие этого файла! Расширение *.lua также является обязательным!

Теперь мы можем написать простейший сценарий. Начнем с того, что зададим время, с которого начнется игра при запуске сценария. Пишем в файле main.lua

```
setTime("17:30")
```

Обратите внимание на синтаксис - время мы указываем в кавычках, так как, с точки зрения игры, астрономическое время это строка. Игра сама преобразует строку в игровое время.

Далее, нам нужно задать поезд, которым мы будем играть. Сделаем это, написав такой код скрипта

```
my_train = TrainData.new()
my_train.name = "ВЛ60пк"
my_train.config = "v160pk-1543"
my_train.traj = "track_a_p1a"
my_train.coord = 1690.0
my_train.dir = 1

setTrain(my_train)
```

Давайте разберем эти "заклинания". my_train - это структура, которая хранит информацию о поезде и его начальном положении. Вы можете дать ей любое имя, но со следующими ограничениями

- имя содержит ТОЛЬКО латиницу!
- имя не должно начинаться с цифры! Цифры далее - разрешены
- пробелы - не допускаются, тире - не допускаются. Используйте вместо них символ подчеркивания "_"

Первая строка

```
my_train = TrainData.new()
```

создает в памяти игры структуру описания вашего поезда. TrainData - стандартное имя этой структуры. new() - оператор, который её создает

Далее

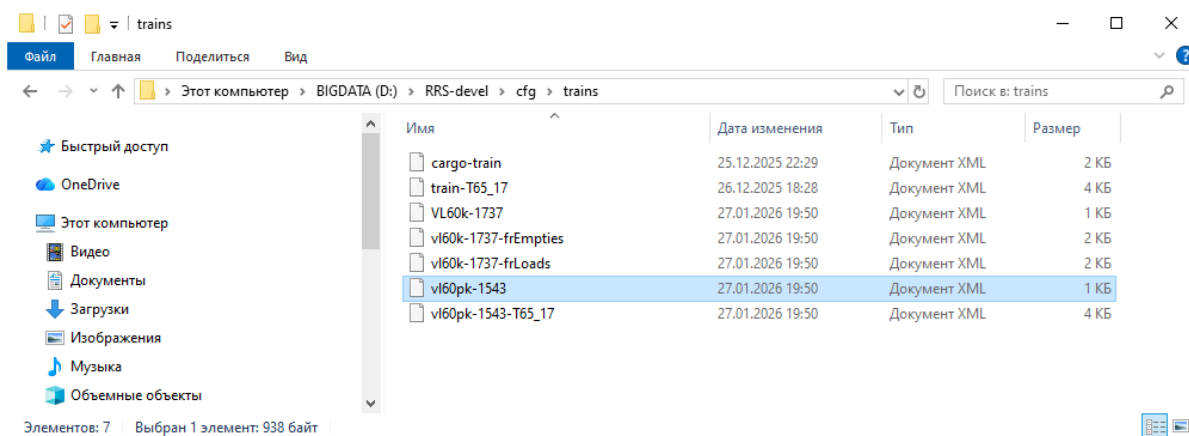
```
my_train.name = "ВЛ60пк"
```

Здесь мы задаем имя нашего поезда. Это строка, поэтому она указана в кавычках! Это имя в дальнейшем используется для идентификации нашего поезда в различных операциях, выполняемых в сценариях.

ВНИМАНИЕ!!! Игра допускает здесь применение кириллических имен, как и в некоторых других случаях, которые мы оговорим позднее.

```
my_train.config = "vl60pk-1543"
```

Задаем имя конфига поезда. Эти конфиги лежат в игре в папке `cfg/trains`. Указанный конфиг - это электровоз ВЛ60пк резервом, вот он

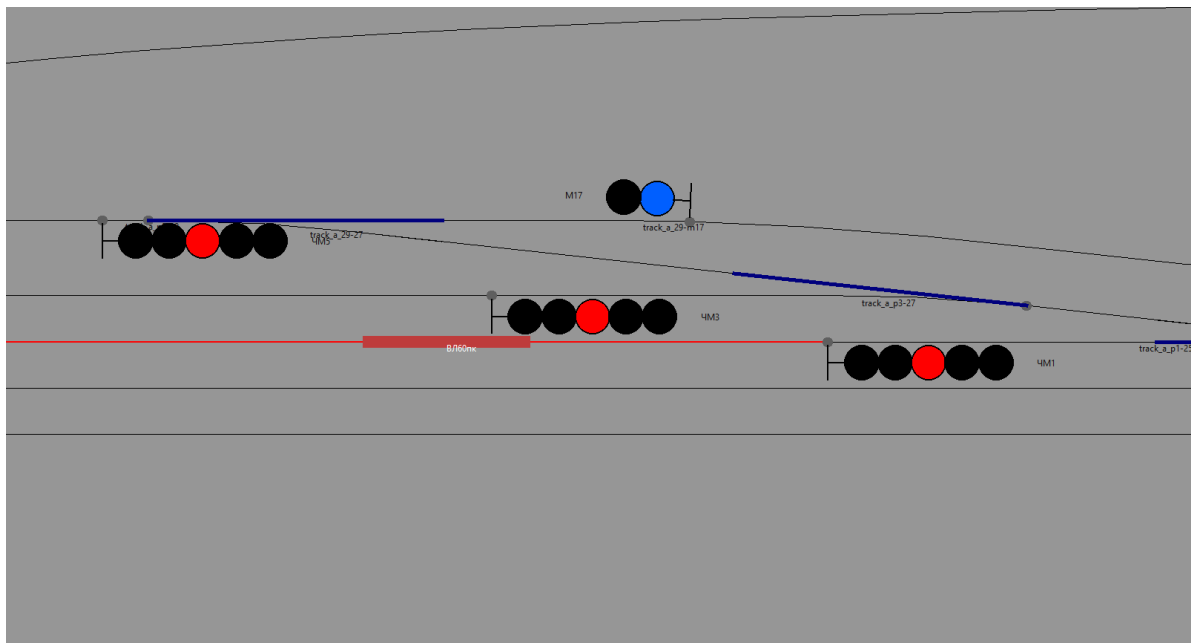


В игру будет загружен поезд, описанный данным конфигом.

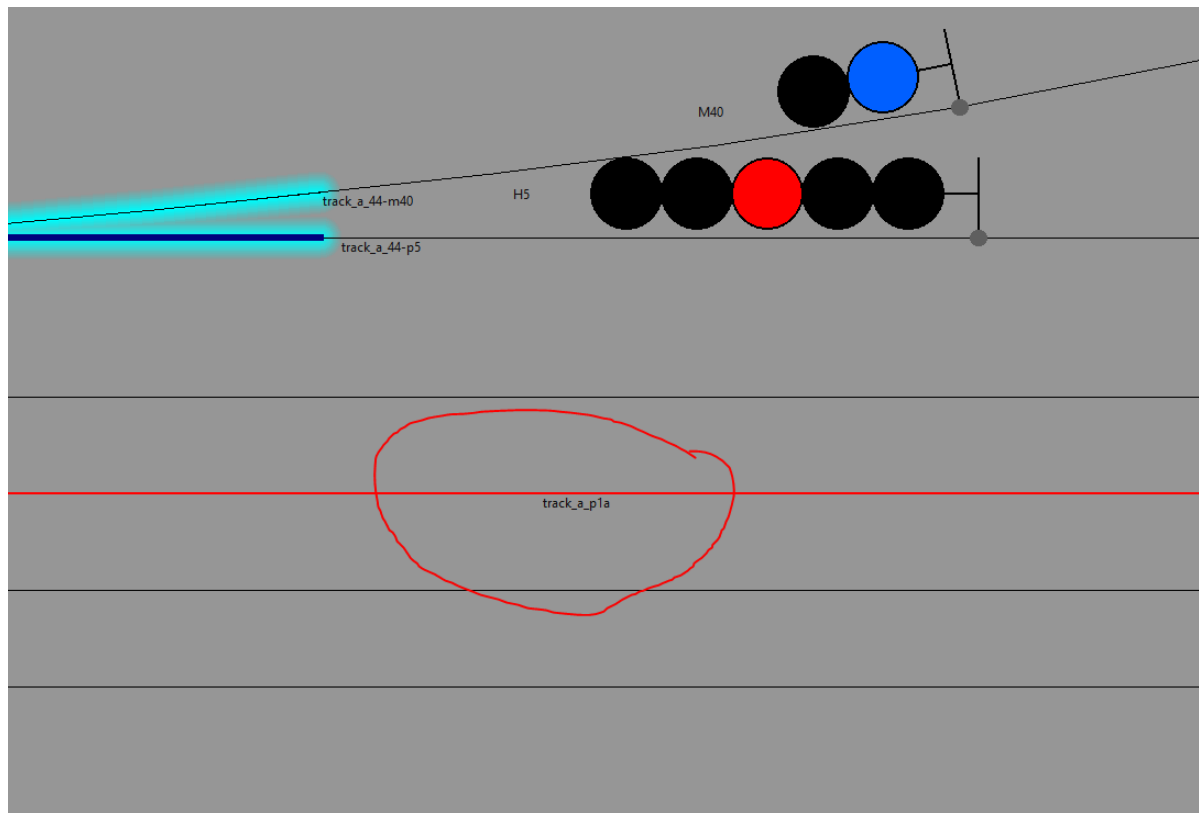
Идем дальше

```
my_train.traj = "track_a_p1a"
```

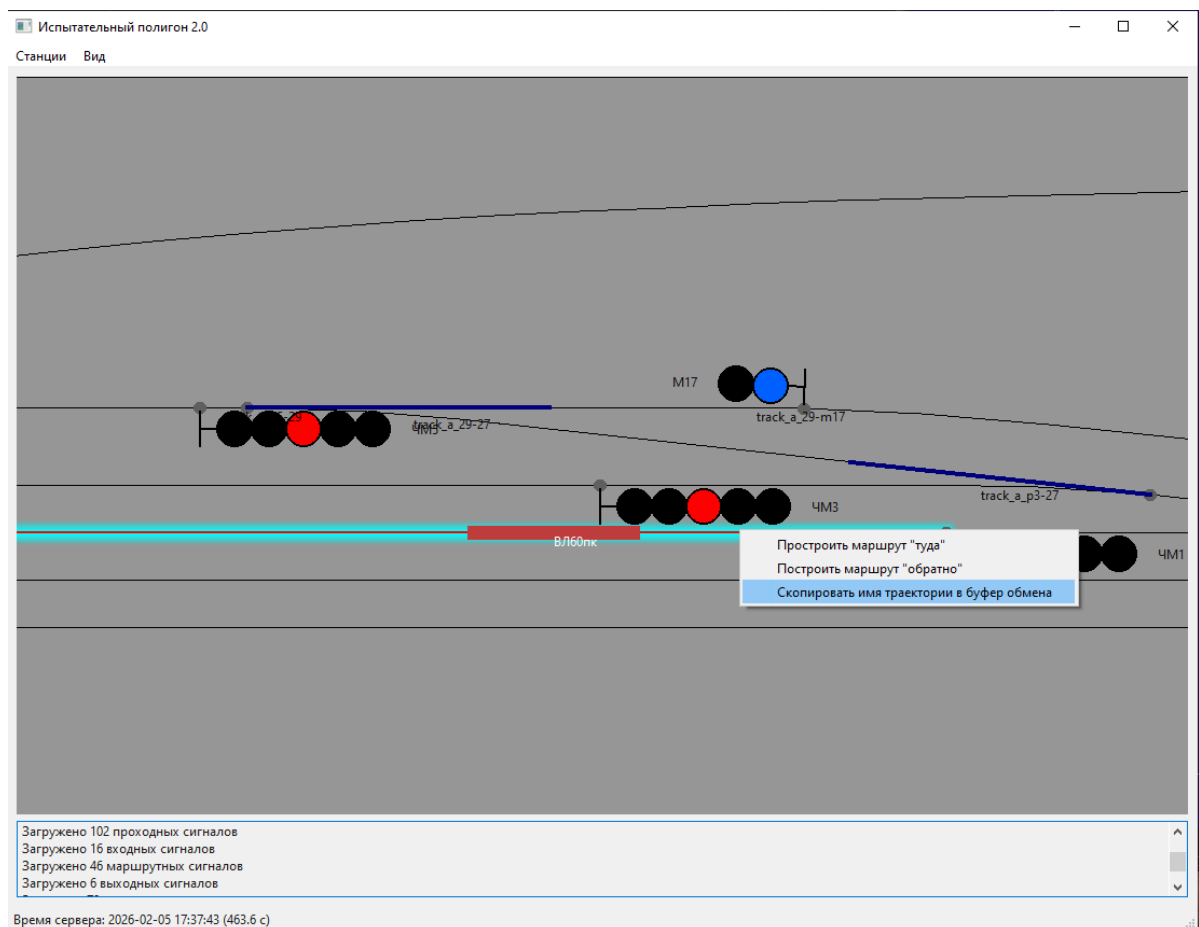
Указываем имя траектории, на которой стоит поезд. Что такое траектория в RRS? Это непрерывный участок пути, разделенный стрелками или изолированными стыками, имеющий собственное имя. В данном случае - это путь 1А на Станции А испытательного полигона. В карте это выглядит так



Как узнать имя траектории? Ну, во-первых, если вы запустите игру в свободном режиме, без сценария, поставив любой поезд куда угодно, вы можете вывести в карте имена всех траекторий через меню "Вид -> Показать имена траекторий". И тогда вы увидите имя интересующей вас траектории, где-то в её середине. Вот так



Но ползать по карте неудобно, искать имя долго. Есть другой способ получить имя интересующей траектории - выбрать ее, наведя курсор, и вызвать меню правой кнопкой

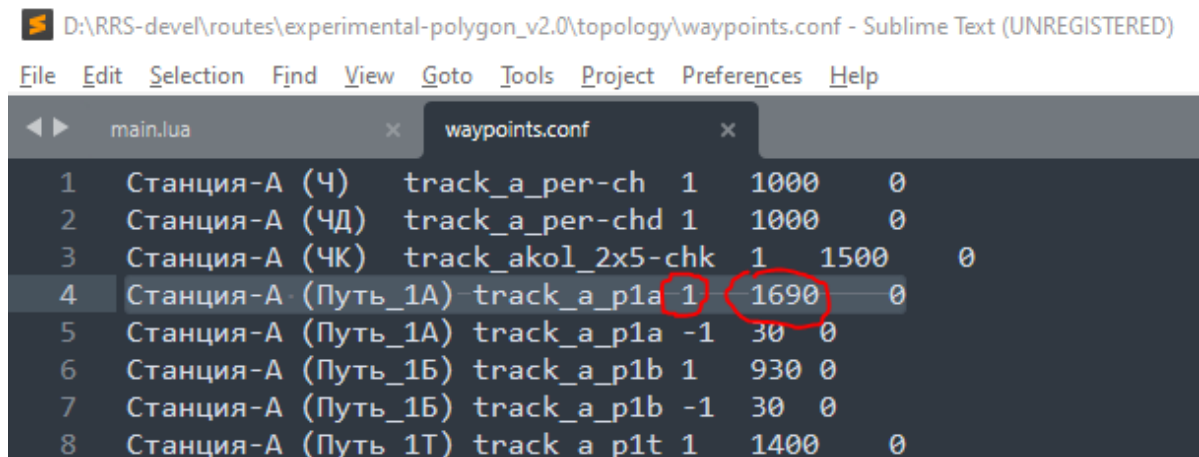


Имя нужной вам траектории будет скопировано в буфер обмена, и в текст сценария вы его можете просто вставить (Ctrl + V / Shift + Insert)

Следующий параметр

```
my_train.coord = 1690.0
```

Координата головы поезда относительно начала траектории, в метрах. Этот параметр можно приблизительно оценить по карте. Другой способ, посмотреть конфиг `topology/waypoints.conf` расположенный в папке с маршрутом. Обычно он включает некоторые траектории на станциях, которые можно использовать в качестве точек старта из лаунчера. Внутри этого файла



перечислены так называемые "путевые точки". Каждая строка включает в себя строку с названием станции и именем пути или выходного светофора с этого пути, имя траектории, направление (1 - в данном случае), и координатой вдоль траектории. Последний параметр

```
my_train.dir = 1
```

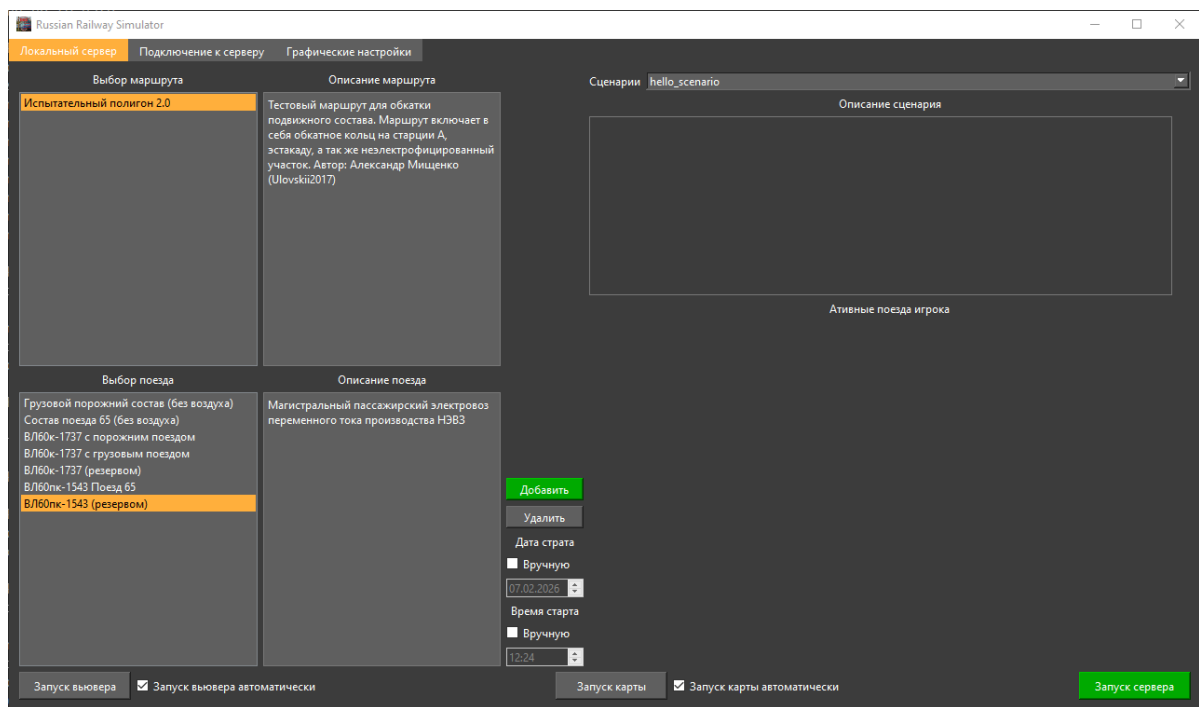
направление головы поезда. В старых терминах маршрутов ZDS 1 - "туда", -1 - "обратно". В терминах RRS - 1 - от начала траектории к концу, -1 - от конца траектории к началу.

Таким образом мы закончили описание поезда и следующей командой устанавливаем поезд в игру

```
setTrain(my_train)
```

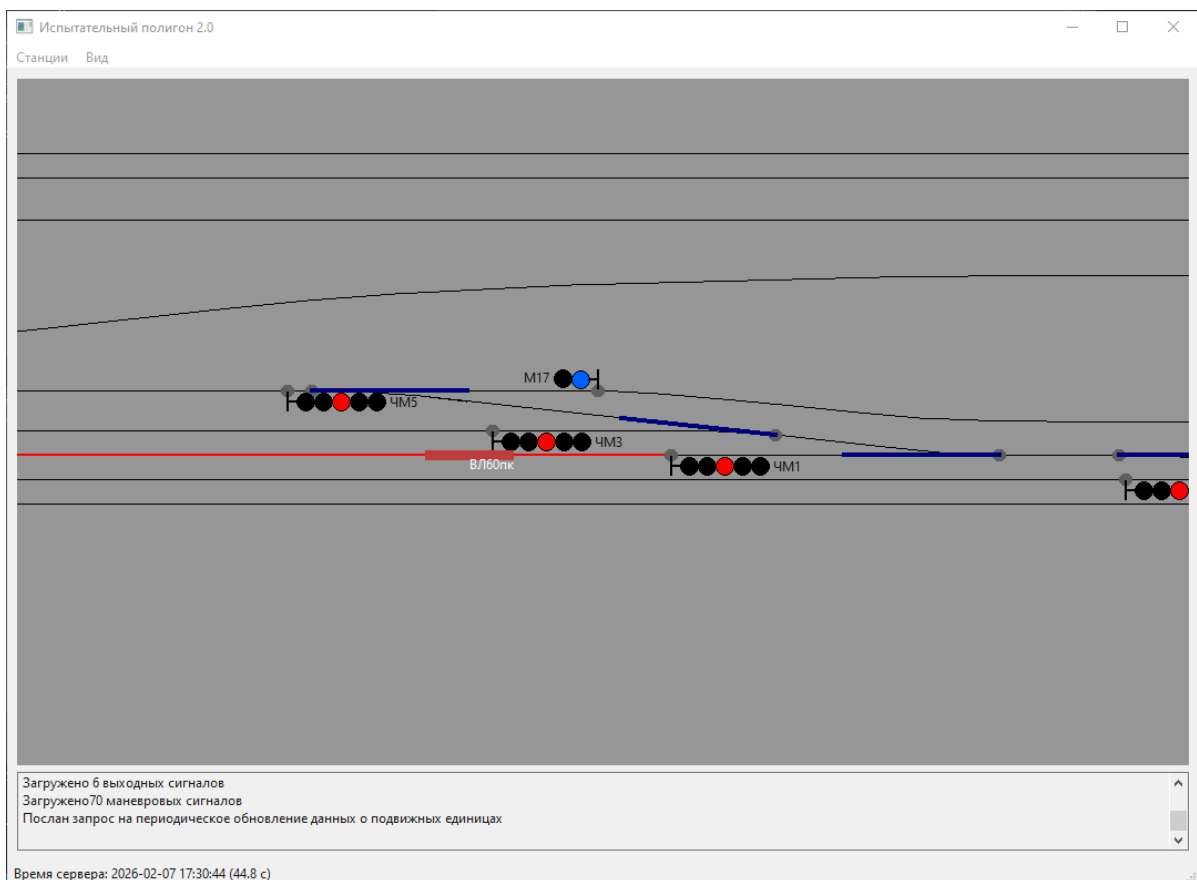
Системная функция `setTrain()` принимает на вход структуру описания поезда.

Мы написали простейший игровой сценарий. Проверим его работу. Запускаем лаунчер игры (`bin/launcher.exe`). Выбираем маршрут и в выпадающем списке "Сценарии" видим наш `hello_scenario`. Выбираем его.

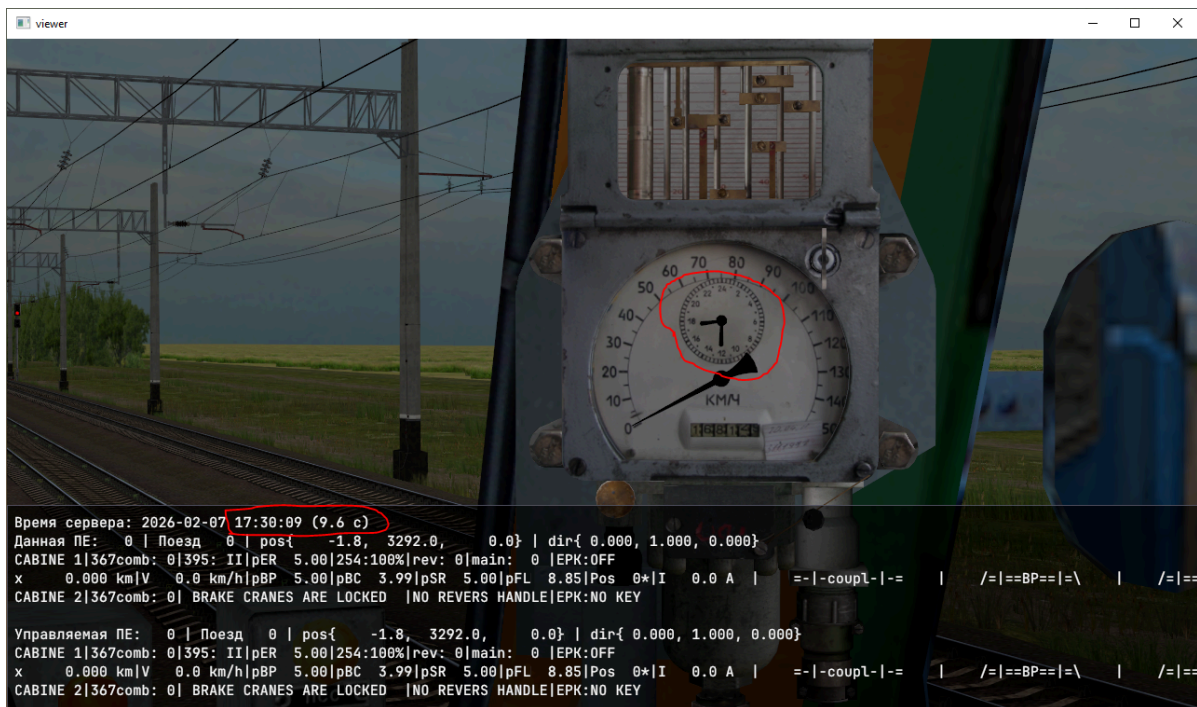


Жмем кнопку "Запуск сервера". Игра запустится, и мы увидим наш поезд, стоящий там, где мы задали в сценарии.





На игровых часах - заданное время, 17:30.



Перед нами простейший сценарий RRS. Но толку от этого сценария довольно мало, давайте теперь попробуем сделать что-то по-настоящему существенное, чего нельзя автоматически выполнить в свободной поездке.

2.2. Построение маршрута отправления. Понятие временного триггера

В реальности поезда отправляются по графику, в определенное графикам время. Дежурный по железнодорожной станции или поездной диспетчер *строят* маршрут отправления, переводя нужные стрелки и открывая сигналы.

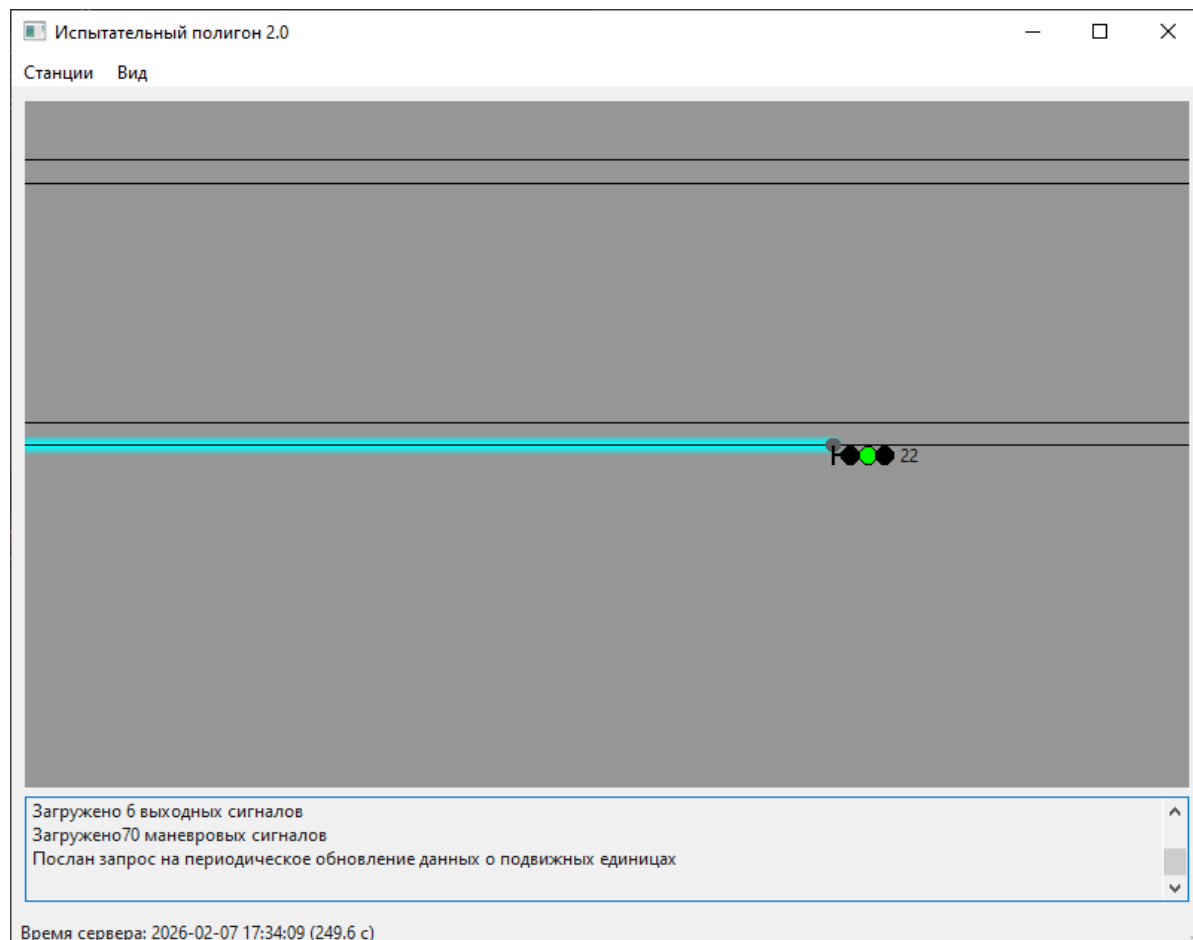
Сценарии RRS позволяют задать такие действия по времени. Для этого предназначены так называемые *временные триггеры* - команда сценария, позволяющая задать произвольное действие, которое выполнится в заданное время. Команда выглядит так

```
setTimeTrigger("время срабатывания", имя_функции_действия)
```

В тексте нашего сценария напомним, например, так

```
setTimeTrigger("17:31", actionBuildTrainRoute("track_a_p1a", "track_a-b_nd-22", 1))
```

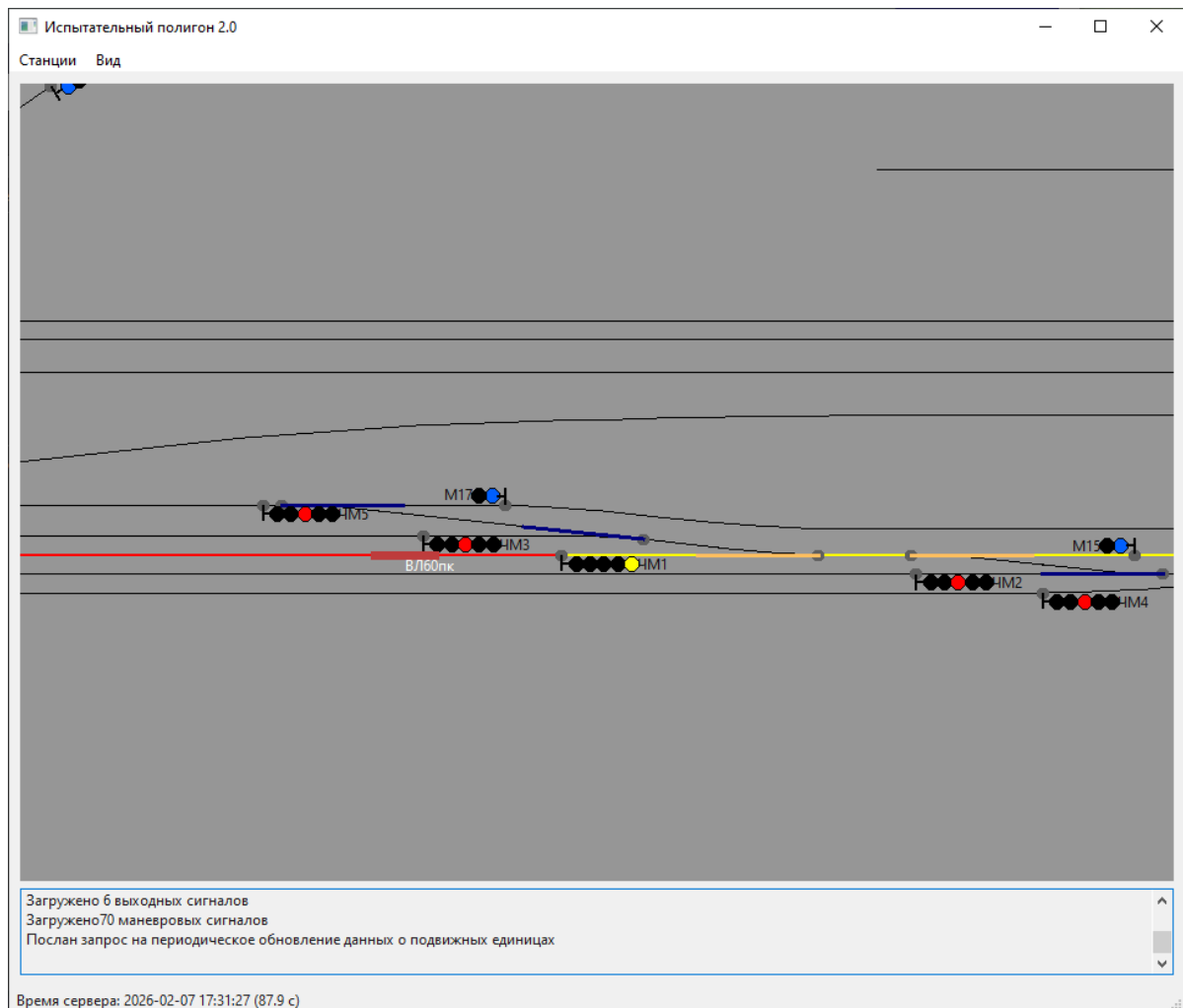
Это означает, что в 17:31 по игровому времени будет построен поездной маршрут от траектории, где находится наш поезд ("track_a_p1a"), до траектории на первом участке удаления от станции А, то есть до вот этой траектории, которая упирается в первый проходной светофор 22 на перегоне.



За построение маршрута отвечает отдельная функция

```
actionBuildTrainRoute("track_a_p1a", "track_a-b_nd-22", 1)
```

Попробуем, что у нас вышло. Перезапускаем игру с нашим сценарием. Ровно в 17:31 открывается маршрутный сигнал ЧМ1, и на карте отображается построенный маршрут отправления



Можно отправляться!

Временной триггер можно задать иначе - установить время относительно времени старта игры. Сделать это можно, указав "+" перед значением времени

```
setTimeTrigger("+00:01", actionBuildTrainRoute("track_a_p1a", "track_a-b_nd-22", 1))
```

Это означает, что маршрут будет построен через 1 минуту после начала игры. Вы можете поэкспериментировать, указав теперь произвольное время в setTime. Маршрут будет построен ровно через минуту от указанного вами времени начала игры.

Для большинства типовых действий игрой предусмотрены стандартные action-функции, полный список которых приведен в [Приложении 2](#).

Таким образом, вот полный текст нашего первого сценария для RRS.

```
setTime("17:30")

my_train = TrainData.new()
my_train.name = "ВЛ60пк"
my_train.config = "v160pk-1543"
my_train.traj = "track_a_p1a"
my_train.coord = 1690.0
my_train.dir = 1

setTrain(my_train)

setTimeTrigger("17:31", actionBuildTrainRoute("track_a_p1a", "track_a-b_nd-22", 1))
```

Согласитесь, выглядит как книга заклинаний. Чтобы сделать этот текст более понятным как для себя, так и для других, можно использовать комментарии. Ведь Lua - полноценный язык программирования и допускает использования комментариев.

В Lua есть два вида комментариев: однострочные и многострочные. Однострочный комментарий начинается с двух идущих подряд тире "--". Например,

```
-- Задаем время начала игры
setTime("17:30")

-- Описываем поезд игрока
my_train = TrainData.new()
my_train.name = "ВЛ60пк"
my_train.config = "v160pk-1543"
my_train.traj = "track_a_p1a"
my_train.coord = 1690.0
my_train.dir = 1

-- Устанавливаем поезд игрока
setTrain(my_train)

-- Строим маршрут отправления в 17:31 с пути 1А станции А
setTimeTrigger("17:31", actionBuildTrainRoute("track_a_p1a", "track_a-b_nd-22", 1))
```

Многострочный комментарий начинается с последовательности символов "--[[\" и оканчивается последовательностью \"]]", например

```
--[[

Мой первый сценарий
для Russian Railway Simulator

]]

-- Задаем время начала игры
setTime("17:30")

-- Описываем поезд игрока
my_train = TrainData.new()
my_train.name = "ВЛ60пк"
my_train.config = "v160pk-1543"
my_train.traj = "track_a_p1a"
my_train.coord = 1690.0
my_train.dir = 1

-- Устанавливаем поезд игрока
setTrain(my_train)

-- Строим маршрут отправления в 17:31 с пути 1А станции А
setTimeTrigger("17:31", actionBuildTrainRoute("track_a_p1a", "track_a-b_nd-22",
1))
```

Крайне рекомендуем использовать комментарии! Это поможет и вам разобраться в том, что вы написали, и другим сценаристам вникнуть в суть происходящего в вашем сценарии. Кроме того, комментарии могут использоваться для того, чтобы сделать неактивной команду или несколько команд сценария, превратив их временно в комментарии (например при отладке, если что-то пошло не так).

2.3. Если что-то пошло не так

Может так получиться, что вы напишете сценарий, например вот так

```
--[[

Мой первый сценарий
для Russian Railway Simulator

]]

-- Задаем время начала игры
setTime("17:30")

-- Описываем поезд игрока
my_train = TraiData.new()
my_train.name = "ВЛ60пк"
my_train.config = "v160pk-1543"
my_train.traj = "track_a_p1a"
my_train.coord = 1690.0
```

```

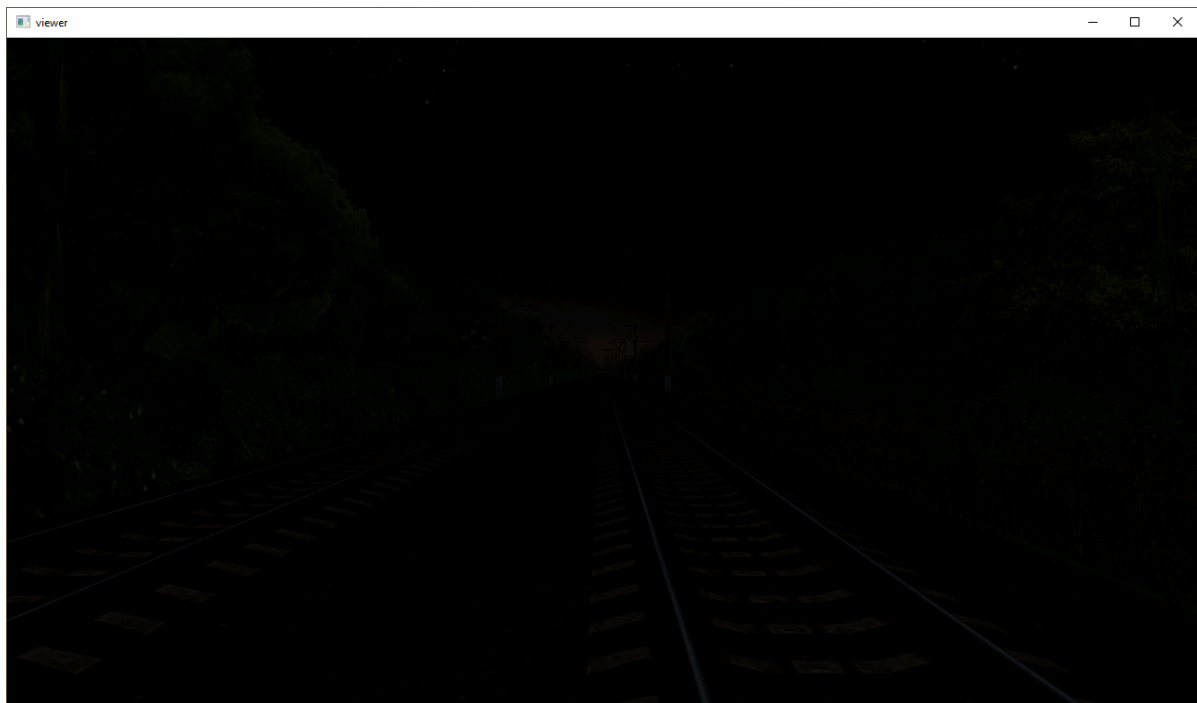
my_train.dir = 1

-- Устанавливаем поезд игрока
setTrain(my_train)

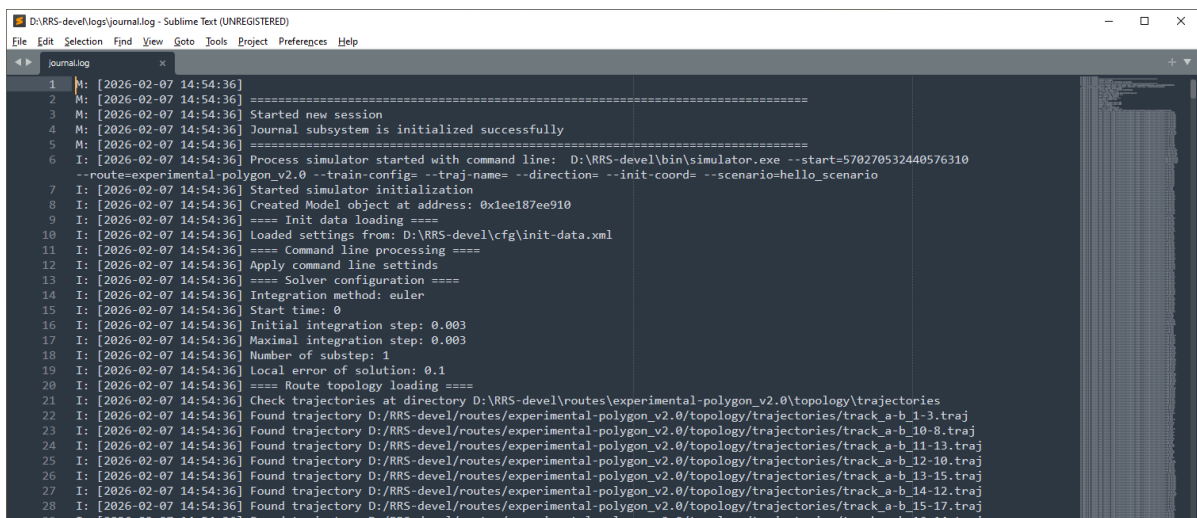
-- Строим маршрут отправления в 17:31 с пути 1А станции А
setTimeTrigger("17:31", actionBuildTrainRoute("track_a_pla", "track_a-b_nd-22",
1))

```

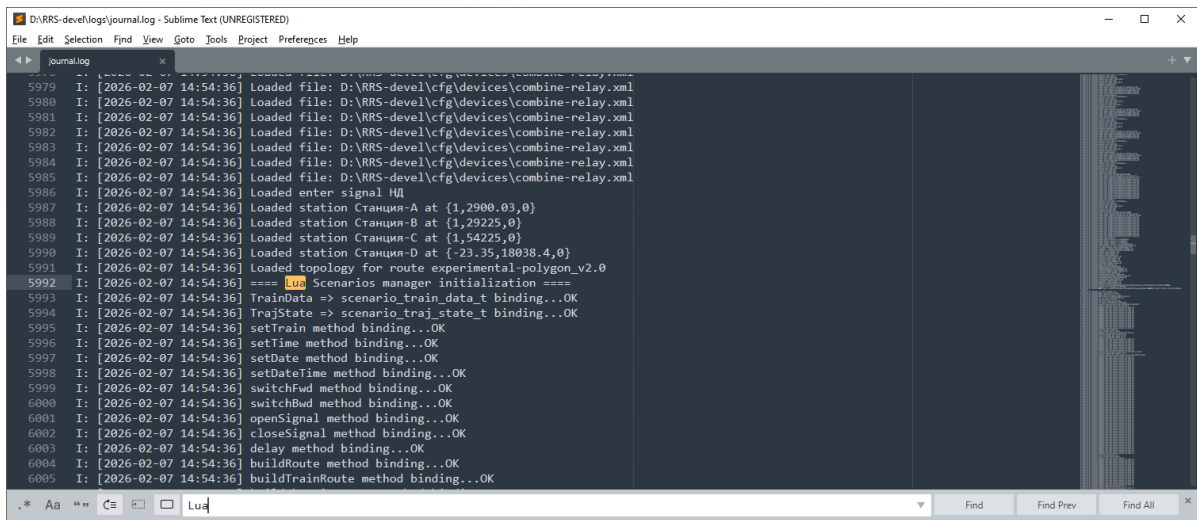
а вместо поезда на станции увидите это



Что делать? То, что вы видите, явный признак того, что где-то в скрипте допущена одна или несколько ошибок. В этом случае вам поможет журнал запуска игры, расположенный в файле logs/journal.log. Откройте его текстовым редактором

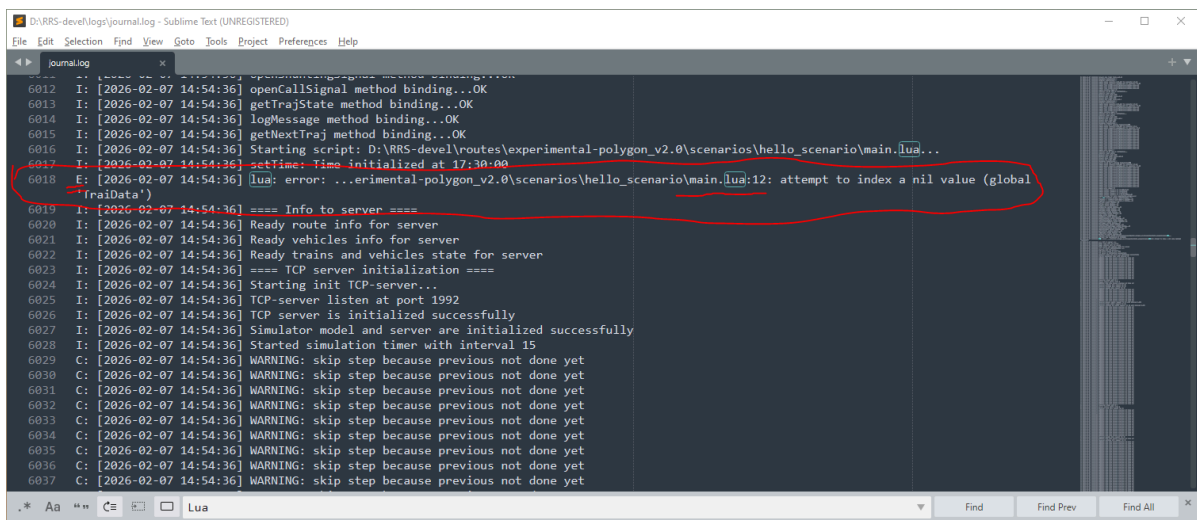


Здесь отражаются очень многие действия, происходящие при старте и работе игры. Тут много всего, поэтому воспользуемся поиском по тексту (Ctrl + F) по ключевому слову "Lua"



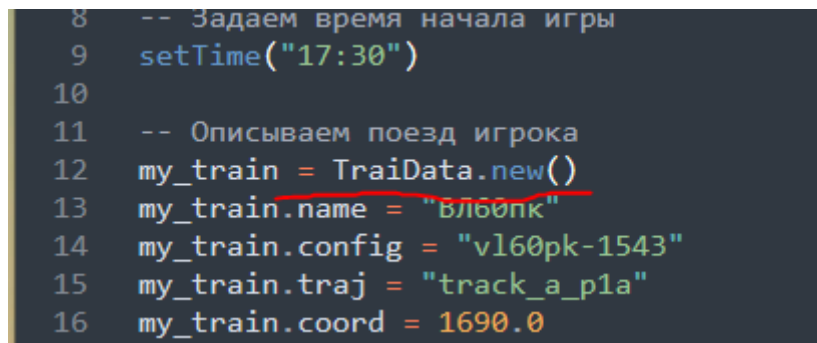
```
5979 I: [2026-02-07 14:54:36] Loaded file: D:\RRS-devel\cfg\devices\combine-relay.xml
5980 I: [2026-02-07 14:54:36] Loaded file: D:\RRS-devel\cfg\devices\combine-relay.xml
5981 I: [2026-02-07 14:54:36] Loaded file: D:\RRS-devel\cfg\devices\combine-relay.xml
5982 I: [2026-02-07 14:54:36] Loaded file: D:\RRS-devel\cfg\devices\combine-relay.xml
5983 I: [2026-02-07 14:54:36] Loaded file: D:\RRS-devel\cfg\devices\combine-relay.xml
5984 I: [2026-02-07 14:54:36] Loaded file: D:\RRS-devel\cfg\devices\combine-relay.xml
5985 I: [2026-02-07 14:54:36] Loaded file: D:\RRS-devel\cfg\devices\combine-relay.xml
5986 I: [2026-02-07 14:54:36] Loaded enter signal HD
5987 I: [2026-02-07 14:54:36] Loaded station Станция-A at {1,2900,0,0}
5988 I: [2026-02-07 14:54:36] Loaded station Станция-B at {1,20225,0}
5989 I: [2026-02-07 14:54:36] Loaded station Станция-C at {1,54225,0}
5990 I: [2026-02-07 14:54:36] Loaded station Станция-D at {-23.35,18038.4,0}
5991 I: [2026-02-07 14:54:36] Loaded topology for route experimental-polygon_v2.0
5992 I: [2026-02-07 14:54:36] ==== Lua Scenarios manager initialization ====
5993 I: [2026-02-07 14:54:36] TrainData => scenario_train_data_t binding...OK
5994 I: [2026-02-07 14:54:36] TrajState => scenario_traj_state_t binding...OK
5995 I: [2026-02-07 14:54:36] setTrain method binding...OK
5996 I: [2026-02-07 14:54:36] setTime method binding...OK
5997 I: [2026-02-07 14:54:36] setDate method binding...OK
5998 I: [2026-02-07 14:54:36] setDateTime method binding...OK
5999 I: [2026-02-07 14:54:36] switchFwd method binding...OK
6000 I: [2026-02-07 14:54:36] switchBwd method binding...OK
6001 I: [2026-02-07 14:54:36] openSignal method binding...OK
6002 I: [2026-02-07 14:54:36] closeSignal method binding...OK
6003 I: [2026-02-07 14:54:36] delay method binding...OK
6004 I: [2026-02-07 14:54:36] buildRoute method binding...OK
6005 I: [2026-02-07 14:54:36] buildTrainRoute method binding...OK
```

Мы нашли место, где выводятся сообщения об обработке сценария игрой. Листаем чуть вниз и находим сообщение об ошибке



```
6012 I: [2026-02-07 14:54:36] openCallSignal method binding...OK
6013 I: [2026-02-07 14:54:36] getTrajState method binding...OK
6014 I: [2026-02-07 14:54:36] logMessage method binding...OK
6015 I: [2026-02-07 14:54:36] getNextTraj method binding...OK
6016 I: [2026-02-07 14:54:36] Starting script: D:\RRS-devel\routes\experimental-polygon_v2.0\scenarios\hello_scenario\main.lua...
6017 I: [2026-02-07 14:54:36] setTime: time initialized at 17:30:00
6018 E: [2026-02-07 14:54:36] Lua: error: ...erimental-polygon_v2.0\scenarios\hello_scenario\main.lua:12: attempt to index a nil value (global TraiData)
6019 I: [2026-02-07 14:54:36] ==== Info to server ====
6020 I: [2026-02-07 14:54:36] Ready route info for server
6021 I: [2026-02-07 14:54:36] Ready vehicles info for server
6022 I: [2026-02-07 14:54:36] Ready trains and vehicles state for server
6023 I: [2026-02-07 14:54:36] ==== TCP server initialization ====
6024 I: [2026-02-07 14:54:36] Starting init TCP-server...
6025 I: [2026-02-07 14:54:36] TCP-server listen at port 1992
6026 I: [2026-02-07 14:54:36] TCP server is initialized successfully
6027 I: [2026-02-07 14:54:36] Simulator model and server are initialized successfully
6028 I: [2026-02-07 14:54:36] Started simulation timer with interval 15
6029 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6030 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6031 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6032 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6033 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6034 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6035 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6036 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
6037 C: [2026-02-07 14:54:36] WARNING: skip step because previous not done yet
```

Здесь написано, что в 12 строке файла main.lua упомянуто несуществующее имя TraiData. Смотрим на двенадцатую строку и действительно видим, что мы допустили опечатку



```
8 -- Задаем время начала игры
9 setTime("17:30")
10
11 -- Описываем поезд игрока
12 my_train = TraiData.new()
13 my_train.name = "ВЛ60пк"
14 my_train.config = "vl60pk-1543"
15 my_train.traj = "track_a_p1a"
16 my_train.coord = 1690.0
```

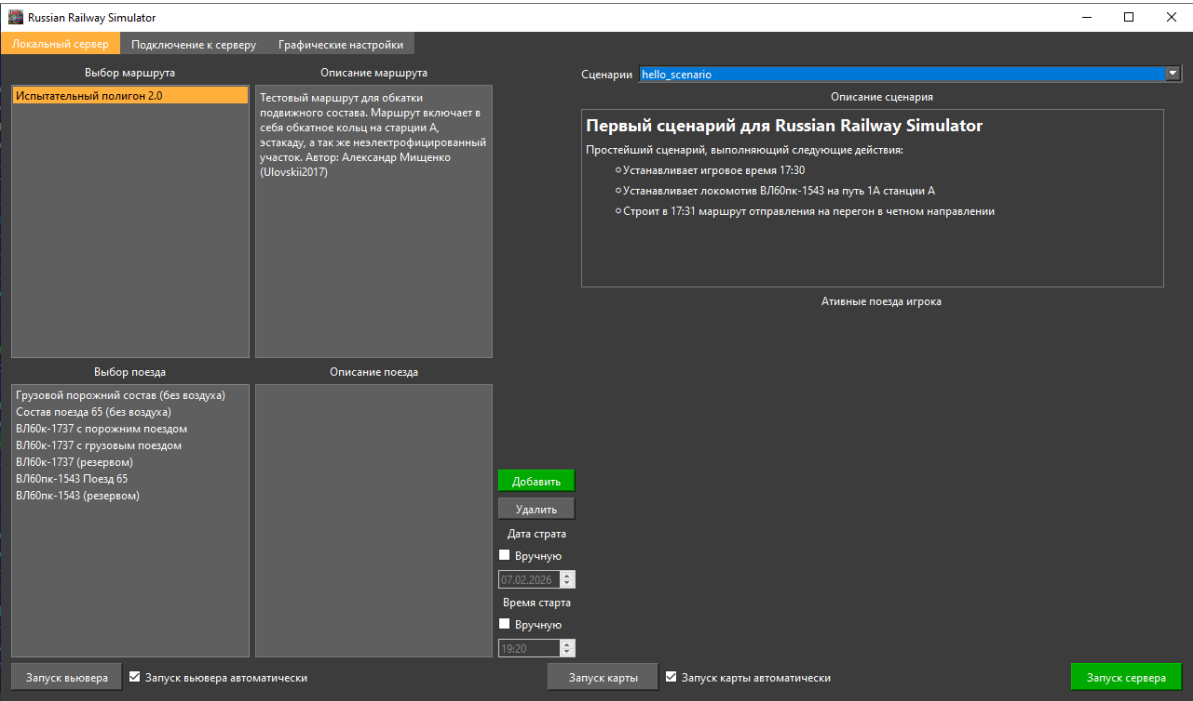
пропустив букву "n" в названии структуры описания поезда TrainData. После исправления этой ошибки игра нормально запустится.

2.4. Файл README.md описания сценария

Согласитесь, название папки со сценарием, которое отображает лаунчер - малоинформативно. Поэтому предусмотрена возможность создавать файл описания сценария.

Этот файл должен иметь название README.md (Это обязательно, с точностью до регистра и расширения!!!).

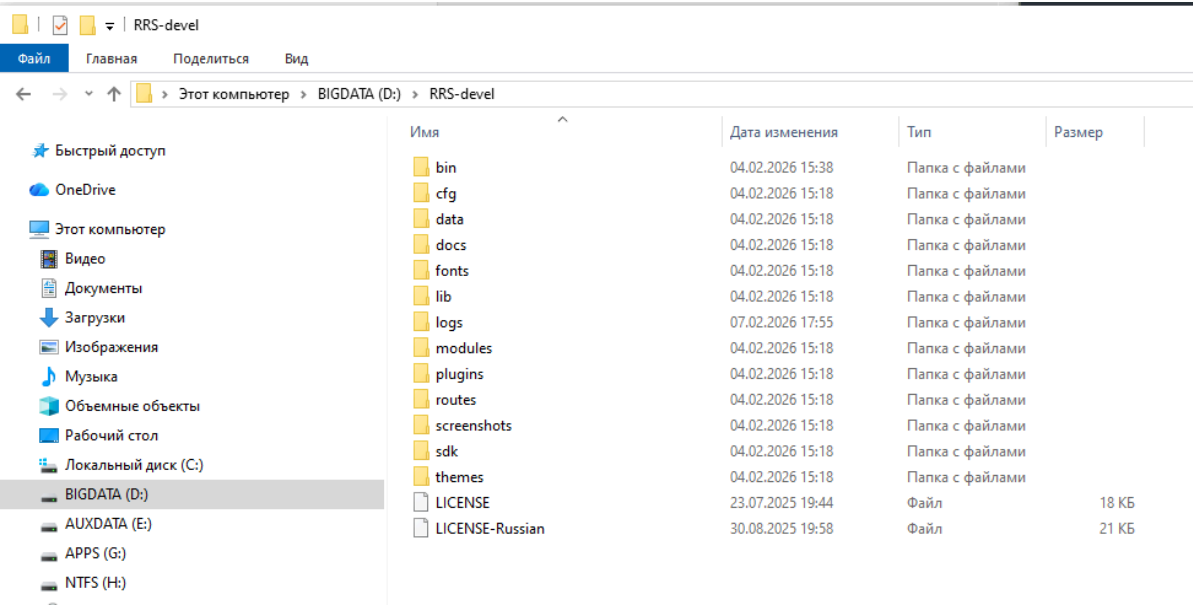
Этот файл текстовый, может содержать произвольный текст в кодировке UTF-8. Он допускает использование разметки Markdown. Если таковой файл присутствует в папке с вашим сценарием, то его содержимое выводится в окно описания сценария при выборе сценария в лаунчере.



2.5. Сценарий, как дополнение к игре. Структура папок

Мы написали простейший сценарий, и продолжим изучение системы сценариев RRS в дальнейшем. Однако, раз речь в данной главе зашла о структуре сценария, то следует сказать, что сценарий - такое же дополнение симулятора, как подвижной состав или маршрут. Потому грамотно разработанный сценарий должен соответствовать структуре дополнения для RRS.

Вспомним структуру каталогов игры



Маршруты располагаются в каталоге routes. Значит, ваш сценарий, в готовом для распространения виде, например для маршрута "Испытательный полигон", должен повторять данную структуру каталогов

```

my_scenario/-
    |-routes/-
        |-experimental-polygon_v2.0/-
            |-scenarios-
                |-my_scenario1
                |-my_scenario2

```

Таким образом, ваш проект сценариев для данного маршрута может содержать несколько папок с разными сценариями.

Например, вам может понадобиться использовать сценарии, которые используют конфигурации поездов, которых нет у игрока. Вы можете заранее сформировать эти поезда, написав их конфиги, и положить в комплект сценария, разместив папке `cfg/trains`, например

```

my_scenario/-
    |-cfg/-
        |-trains/-
            |-train104.xml
            |-train105.xml
    |-routes/-
        |-experimental-polygon_v2.0/-
            |-scenarios-
                |-my_scenario1
                |-my_scenario2

```

Здесь `train104.xml` и `train105.xml` - конфиги поездов, которые вы составили сами и используете в сценарии.

Можно пойти дальше, например, ваш поезд `train104.xml` использует нестандартный конфиг для электровоза ВЛ60пк, которого нет в игре. Вы можете, учтя структуру каталогов игры, поставить такой конфиг вместе со сценарием. Не буду подробно описывать это здесь, но такая возможность существует, и её реализация требует более глубокого понимания структуры игры, которое можно почерпнуть из изучения документации на сайте <https://rusrailsim.ru>

3. Более сложные команды сценариев.

Позиционные триггеры

Выполнять произвольные действия в заданное время - это очень полезная функция. Но некоторые события крайне трудно или вовсе невозможно привязать к заданному времени.

В прошлой главе мы строили маршрут отправления для одного единственного поезда. Это действие логично привязать ко времени. А как быть с маршрутом прибытия? Ведь наш поезд, хоть он и должен ехать по графику, может и не успеть подъехать к станции в заданное время. Он может приехать вовремя, но может приехать позже или приехать раньше. Как быть?

Для того, чтобы ответить на этот вопрос, нам нужно поставить себя на место дежурного по железнодорожной станции. Когда он начинает готовить маршрут для прибывающего поезда? Обычно, когда поезд появляется на втором участке приближения, то есть находится за два блок-участка до входного светофора.

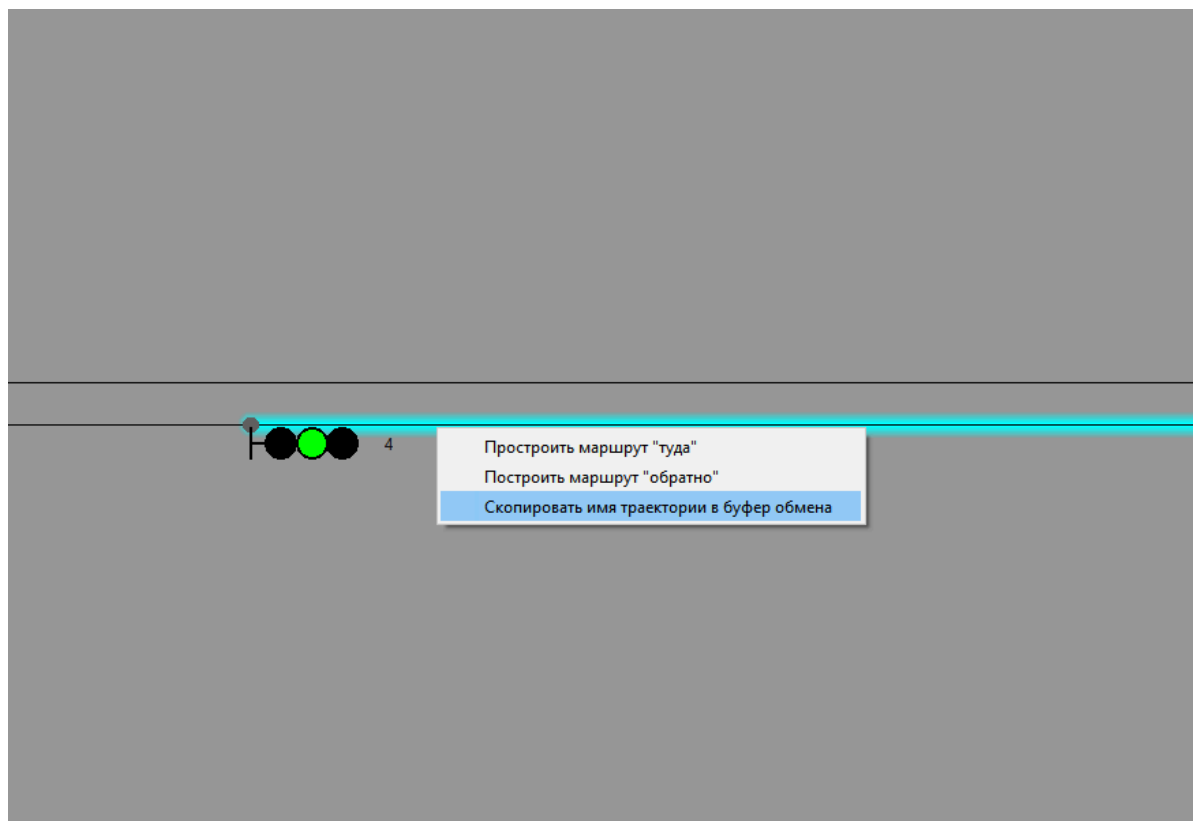
Было бы хорошо, чтобы сценарий мог построить нам маршрут, когда наш поезд займет второй участок приближения. Можно ли это сделать в RRS? Разумеется!

Каждый раз, когда какой-либо поезд занимает или освобождает какую-либо траекторию, внутри игры генерируется событие "изменение состояния траектории". И мы можем создать для симулятора так называемый *позиционный триггер*, который выполнит нужное нам действие, когда будет занята или освобождена интересующая нас траектория.

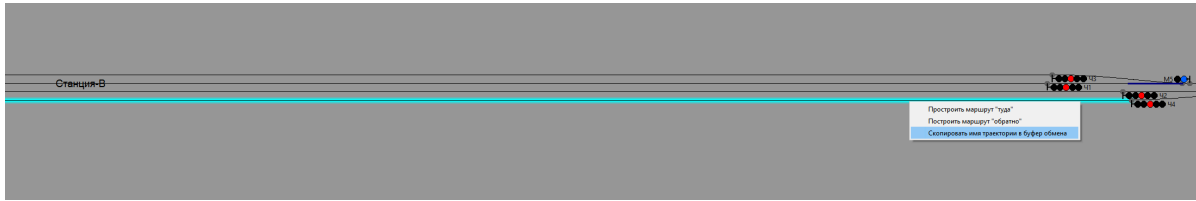
Движок игры предоставляет для этого очень гибкие механизмы, но сначала мы рассмотрим простейшие.

3.1. Строим маршрут приема поезда

Итак, в прошлой главе мы отправили поезд на перегон. Допустим, он почти достиг следующей станции, оказавшись на втором приближении к ней, то есть он занял блок участок за светофором 4.



Скопируем имя этой траектории в буфер обмена. Имя этой траектории "track_a-b_4-2". Когда поезд займет её, необходимо построить маршрут приема на следующую станцию, она у нас Станция В. Например на 4 боковой путь. Посмотрим на карте, как называется этот путь.



Имя целевой траектории "track_b_p4". Отлично, теперь нужна начальная точка, откуда будет построен маршрут. В качестве нее может выступать та траектория, которую мы заняли, то есть "track_a-b_4-2".

Для установки маршрута создадим позиционный триггер такой командой

```
-- Строим маршрут приема на 4 путь станции В по факту занятия второго четного
приближения
setOnTrajBusyTrigger("track_a-b_4-2", actionBuildTrainRoute("track_a-b_4-2",
"track_b_p4", my_train.dir))
```

Это означает следующее: если занята траектория "track_a-b_4-2", необходимо выполнить действие, указанное в следующем параметре функции setOnTrajBusyTrigger(). В роли этого действия выступает стандартная и уже знакомая нам функция actionBuildTrainRoute() (смотрите [главу 2](#)), которая строит поездной маршрут от одной траектории до другой в заданном направлении.

Обратите внимание, что вместо 1 или -1 мы указали направление, передав в action-функцию my_train.dir. Это логично - ведь мы уже задали для нашего поезда его начальное направление движения, и можем воспользоваться им, так как оно не менялось.

Таким образом, как только наш поезд заедет на второй участок приближения, будет построен маршрут его приема на 4-ый путь станции В!

Теперь текст нашего сценария выглядит так

```
--[[

Мой первый сценарий
для Russian Railway Simulator

]]

-- Задаем время начала игры
setTime("17:30")

-- Описываем поезд игрока
my_train = TrainData.new()
my_train.name = "ВЛ60пк"
my_train.config = "v\l60pk-1543"
my_train.traj = "track_a_p1a"
my_train.coord = 1690.0
my_train.dir = 1

-- Устанавливаем поезд игрока
```

```
setTrain(my_train)
```

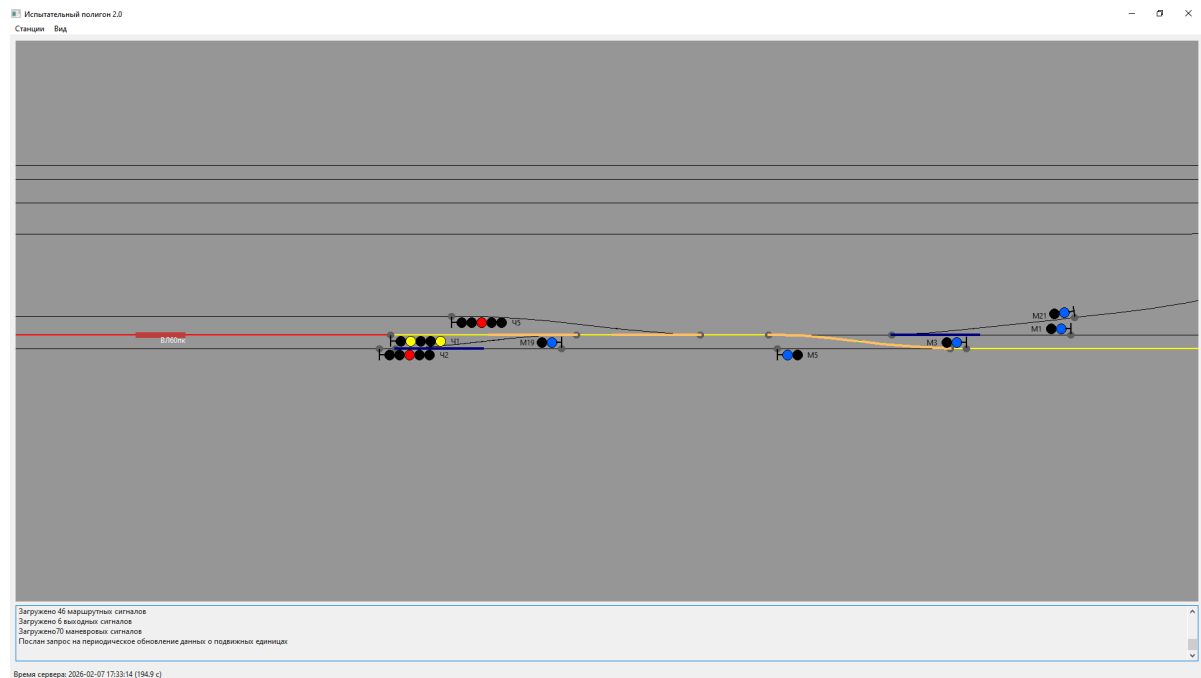
-- Строим маршрут отправления в 17:31 с пути 1А станции А

```
setTimeTrigger("17:31", actionBuildTrainRoute("track_a_p1a", "track_a-b_nd-22",  
my_train.dir))
```

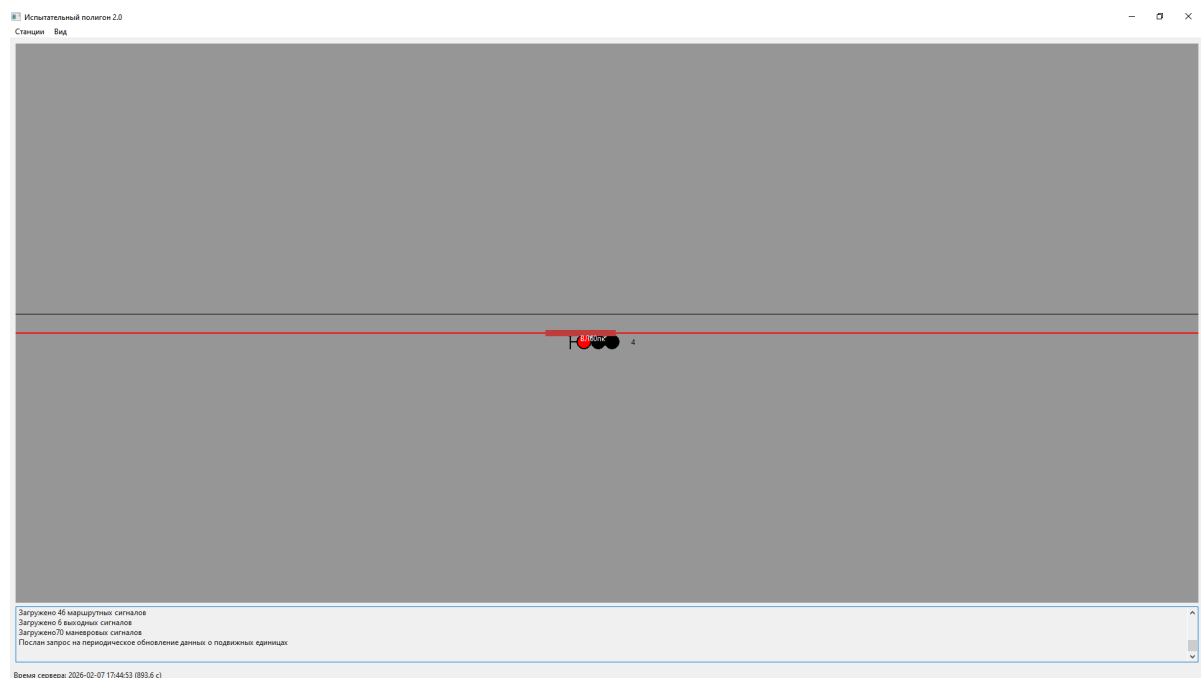
-- Строим маршрут приема на 4 путь станции В по факту занятия второго четного приближения

```
setOnTrajBusyTrigger("track_a-b_4-2", actionBuildTrainRoute("track_a-b_4-2",  
"track_b_p4", my_train.dir))
```

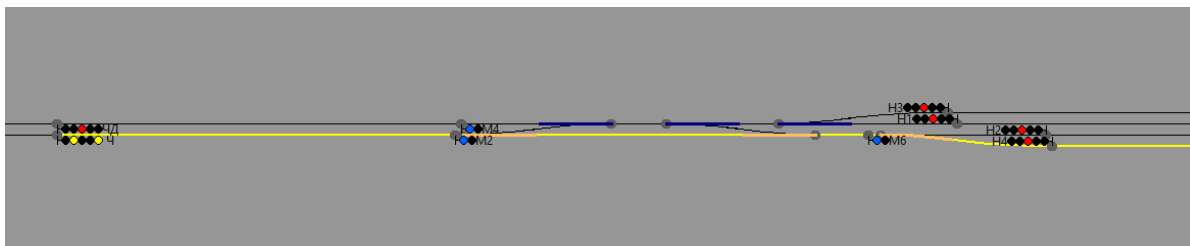
В 17:31 мы отправляемся и выходим на перегон.



Занимаем второй участок удаления со стороны четной горловины станции В



Нам строят маршрут приёма на 4 путь



Не важно, соблюдает наш поезд график или нет. Маршрут приема будет построен ровно тогда, когда будет занята траектория, указанная в функции установки триггера!

Не важно, какой длины наш поезд. Симулятор считает траекторию занятой, когда на неё въезжает голова поезда. Симулятор считает траекторию свободной только тогда, когда её покидает последняя единица подвижного состава, входящая в поезд. Все это автоматически учитывается игрой. Для сценариста предоставляется факт - траектория занята/траектория свободна. И на этот факт можно прикрепить совершенно произвольное действие.

3.2. Строим маршрут приема для конкретного поезда

А если нам нужно построить конкретный маршрут для конкретного поезда. Скажем, скорый поезд 34 мы должны пропустить по второму главному пути? Для этого нужен триггер, который построит маршрут по занятию участка приближения конкретным поездом.

Для начала вспомним, что у нашего поезда есть имя

```
my_train.name = "вл60пк"
```

Присвоим ему имя "34"

```
my_train.name = "34"
```

Поменяем конфиг поезда, чтобы у нас были вагоны, например возьмем стандартный конфиг для пассажирского поезда из комплекта симулятора

```
my_train.config = "vl60pk-1543-T65_17"
```

Вообще, напомним другой сценарий, руководствуясь знаниями, полученными в [главе 2](#).

```
-- Задаем время начала игры
setTime("17:30")

-- Описываем поезд игрока
train34 = TrainData.new()
train34.name = "34"
train34.config = "vl60pk-1543-T65_17"
train34.traj = "track_a_p2b"
train34.coord = 1040.0
train34.dir = 1

setTrain(train34)

-- Строим маршрут отправления в 17:31 с пути 2Б станции А
setTimeTrigger("17:31", actionBuildTrainRoute(train34.traj, "track_a-b_nd-22",
train34.dir))
```

```
-- Строим маршрут пропуска поезда 34 по 2 главному пути станции В  
setOnTrajBusyByTrainTrigger("track_a-b_4-2", "34",  
actionBuildTrainRoute("track_a-b_4-2", "track_b-c_nd-22", train34.dir))
```

Тут важна функция `setOnTrajBusyByTrainTrigger()`. Первым параметром она задает ту траекторию, которую надо занять, а вторым - имя поезда, который должен это сделать. В качестве действия - построение сквозного маршрута по 2 пути станции В, конечная траектория которого - первый четный участок удаления (смотрим по карте!)

Обратите внимание и на то, что в качестве начальной траектории маршрута отправления мы указали не конкретное имя, а параметр структуры, описывающей поезд `train34.traj`. Ведь мы уже задали ту траекторию, где находится поезд, нам незачем писать имя этой траектории повторно.

3.3. Имя поезда может измениться!

Как это? Да очень просто - в RRS это возможно. Например, когда вы начинаете игру на одиночном локомотиве и собираетесь, совершив маневры по станции, прицепиться к составу. Тогда в начале игры у вас было два поезда, а после прицепки - будет один поезд.

В этом случае имена обоих ранее существовавших поездов сбросятся и станут равны "0000".

Аналогично - после отцепки от состава, у вас был один поезд, а станет - два. Имя старого поезда будет сброшено, а состав и отцепившийся локомотив получают имя "0000".

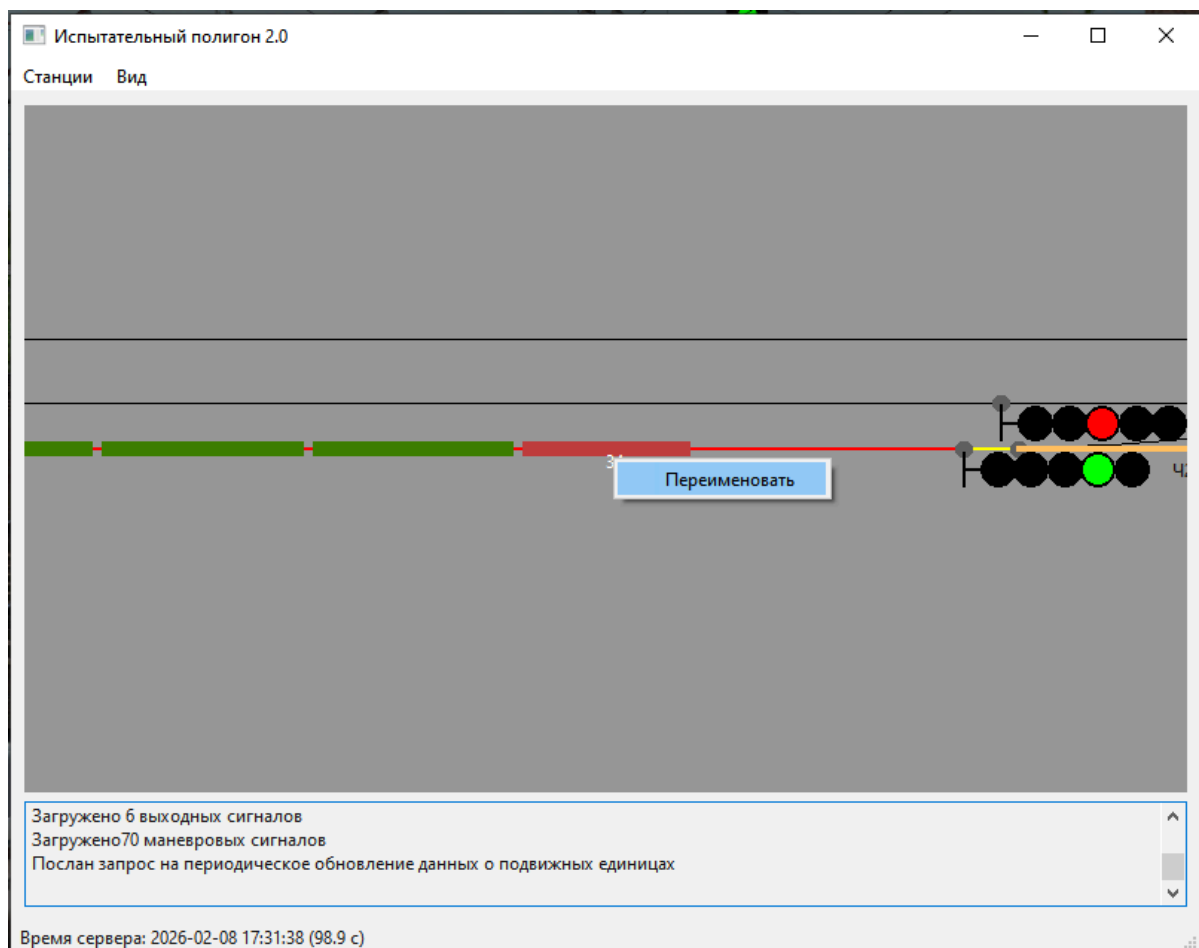
Но сценарий должен точно знать имя поезда, как мы увидели из предыдущего примера. Как быть?

Да так же, как и в жизни! Присвоить поезду нужное имя.

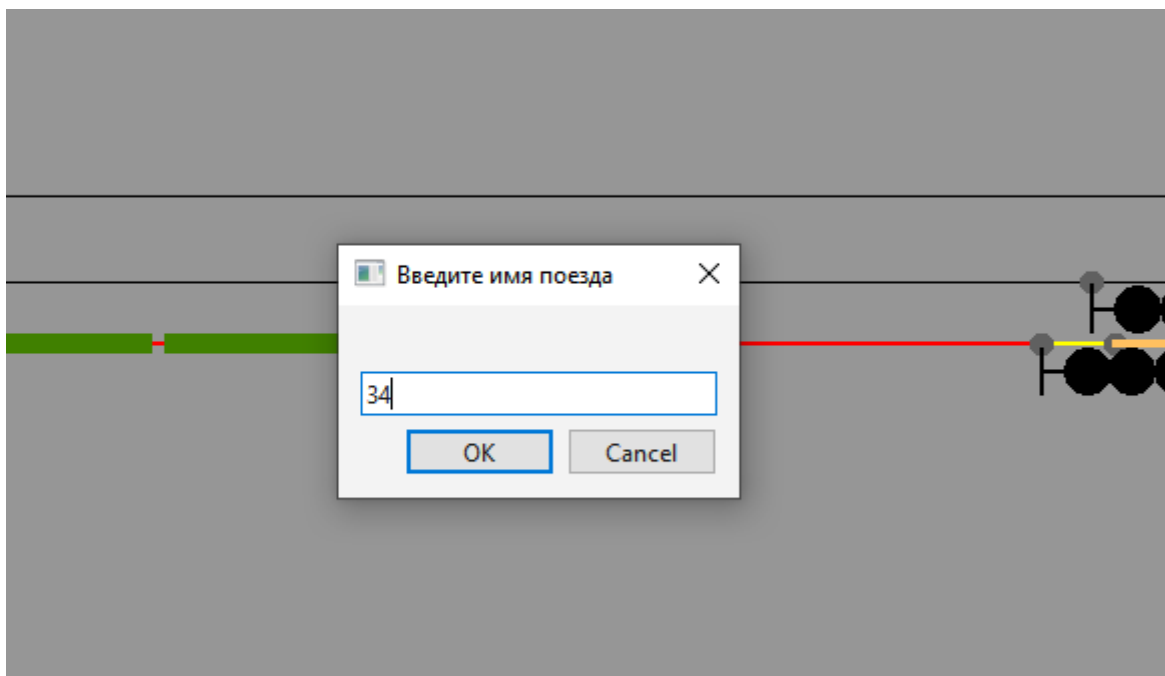
Это можно сделать прямо в кабине локомотива. Нажмите F8. В появившемся окне введите новое имя (допускается латиница и цифры!) и нажмите Enter. Поезд получит новое имя.



Это можно сделать из карты. Щелкните правой кнопкой по старому имени поезда и в выпавшем контекстном меню выберите "Переименовать".



В появившемся диалоговом окне надо ввести новое имя, которое тут же отобразится в карте



Если вы играете с друзьями, переименование поездов из карты может взять на себя ваш товарищ по игре, играющий роль дежурного по станции или диспетчера.

ВНИМАНИЕ!!! Если ваш сценарий предусматривает выполнение маневров с прицепкой или отцепкой локомотива - предупредите игрока о том, когда и какое имя нужно дать поезду или локомотиву! Это можно сделать в описании сценария в файле README.md.

Такая механика дает возможность заранее планировать действия в сценарии, какие бы маневры не совершались фактически.

3.4. Прочие позиционные триггеры

Все имеющиеся стандартные позиционные триггеры описаны в [приложении 3](#). Разумеется, аналогично занятию траектории, предусмотрены триггеры, срабатывающие и на её освобождение как произвольным поездом, так и поездом с конкретным именем.

Важно отметить, что эти триггеры являются одноразовыми - после срабатывания они удаляются из игры. Для большинства стандартных ситуаций это подходит, но возможно, что сценаристу потребуется более сложная механика работы с триггерами.

Симулятор обеспечивает произвольное комбинирование стандартных триггеров. Например, можно создать триггер, действием для которого будет установка другого триггера. Это обеспечивают как временные, так и позиционные триггеры. Существует специальный временной триггер `setPostEventTimeTrigger()`, который срабатывает через заданный промежуток времени после своей установки. То есть вы можете выполнить любое действие через определенное время после наступления события, определенного другим триггером.

Для того, чтобы использовать эти механики, необходимо писать собственные action-функции, до сих пор мы пользовались стандартными action-функциями, перечисленными в [приложении 2](#). Однако этот вопрос оставим в качестве материала для следующей главы.

Кроме того, разработчик сценария имеет возможность самостоятельно управлять временем жизни триггера. Это мы тоже рассмотрим в следующей главе, посвященной более сложным механикам сценариев. А пока рассмотрим один интересный стандартный триггер.

3.5. Автоматический пропуск поездов по станции

Представим себе ситуацию, когда у нас есть небольшая промежуточная станция, на которой в нашем сценарии не предусматривается принятие или отправление поездов. Но для того, чтобы поезда беспрепятственно проследовали через неё, нам необходимо прописать для них маршруты пропуска, для каждого.

Для облегчения решения этой задачи предусмотрена специальная триггерная функция, обеспечивающая автоматическое построение поездных маршрутов в заданном направлении. Эта функция описана в [приложении 3](#), и называется она `setAutoApproachTrigger()`.

В качестве параметров эта функция принимает начальную траекторию, конечную траекторию и направление (1 или -1). Она выполняет построение маршрута при занятии любым поездом начальной траектории. Маршрут строится до конечной траектории в заданном направлении.

Особенностью этой функции является то, что будучи один раз заданной в сценарии, она работает в течение всей игры.

Для примера решим такую задачу. На испытательном полигоне есть станция В. Организуем на ней двухсторонний автоматический сквозной пропуск поездов.

Мы же знаем, как называются участки приближения и удаления у станции В в четном направлении. Тогда вот такая команда в сценарии

```
setAutoApproachTrigger("track_a-b_4-2", "track_b-c_nd-22", 1)
```

обеспечит построение маршрутов пропуска в четном направлении для всех поездов, которые вступают на траекторию "track_a-b_4-2". Для нечетного направления справимся с картой насчет участков приближения и удаления. Тогда такая команда создаст триггеры для автоматического пропуска поездов в нечетном направлении

```
setAutoApproachTrigger("track_b-c_1-3", "track_a-b_21-chd", -1)
```

Создадим под это отдельный сценарий, для ускорения демонстрации расположив два поезда на перегоне по обе стороны от станции. Назовем сценарий auto_approach, а текст его основного скрипта будет таким

```
setTime("12:00")

-- Устанавливаем поезд с четного направления
train1 = TrainData.new()
train1.name = "34"
train1.config = "v160pk-1543-T65_17"
train1.traj = "track_a-b_6-4"
train1.coord = 1900.0
train1.dir = 1

setTrain(train1)

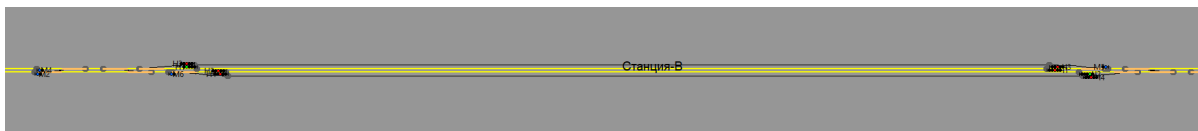
-- Устанавливаем поезд с нечетного направления
train2 = TrainData.new()
train2.name = "35"
train2.config = "v160pk-1543-T65_17"
train2.traj = "track_b-c_3-5"
train2.coord = 50.0
train2.dir = -1

setTrain(train2)

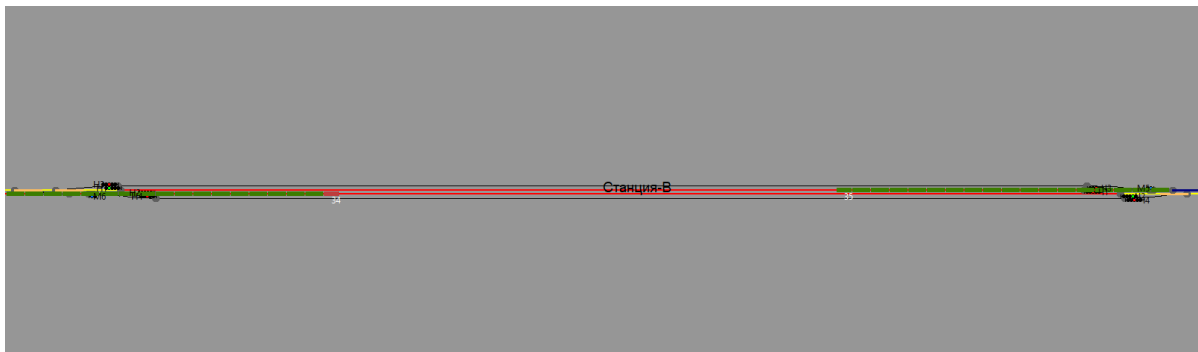
-- Автоматический пропуск поездов в четном направлении
setAutoApproachTrigger("track_a-b_4-2", "track_b-c_nd-22", 1)

-- Автоматический пропуск поездов в нечетном направлении
setAutoApproachTrigger("track_b-c_1-3", "track_a-b_21-chd", -1)
```

Как только эти поезда тронутся с места и займут соответствующие участки приближения, будут построены два маршрута пропуска для них



Эти поезда закономерным образом встретятся и разойдутся по станции





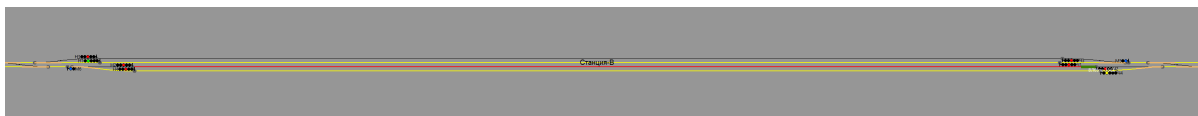
А триггеры останутся активными и будут готовить маршрут пропуска для других поездов, приближающихся к данной станции.

Этот автомат учитывает, что некоторые пути станции могут быть заняты. Расположим на 2 пути электровоз, добавив его в качестве третьего поезда в наш сценарий

```
train3 = TrainData.new()
train3.name = "ВЛ60пк-1543"
train3.config = "v160pk-1543"
train3.traj = "track_b_p2"
train3.coord = 1690.0
train3.dir = 1

setTrain(train3)
```

Алгоритм построения маршрута обойдет занятый путь по незанятому 4-му пути.



Таким образом, с помощью стандартных триггеров системы сценариев можно скриптом из 30 строк организовать достаточно сложное поведение железнодорожной автоматики в игре.

4. Продвинутые приемы разработки сценариев

То, что мы рассмотрели в предыдущих главах, касается базовых команд сценариев, выполняющих достаточно элементарные операции. Основываясь на этих знаниях, можно создавать достаточно сложные сценарии. Однако в полной мере возможности сценарного движка раскрываются при создании собственных функций.

Для этого необходимо более детальное погружение в алгоритмы работы сценариев в RRS.

4.1. Порядок выполнения скрипта сценария

Как выполняется скрипт сценария? Тот файл, что мы создавали на протяжении трех глав, `main.lua`, загружается при запуске игры и выполняется **полностью** от начала и до конца. Все записанные туда команды так или иначе воздействуют на движок игры, заставляя её выполнять определенные действия. Одни команды выполняются сразу же, другие - регистрируются симулятором и ждут своего часа. Разберем это подробнее.

1. Команды, выполняемые при загрузке игры

Это все, что касается начальной инициализации поездки

- `setTime`, `setDate`, `setDateTime` - устанавливают дату при запуске игры
- `setTrain` - устанавливает заданный поезд в заданное место карты. Это тоже выполняется при запуске игры

Все остальные команды имеют отложенное действие.

2. Команды, выполняемые в процессе игры

Команды установки триггеров добавляют триггер в общий список триггеров, и в процессе исполнения игры происходит периодическая проверка всех триггеров на соответствие заданных триггером условий. При совпадении этих условий у какого-то из триггеров (например наступление заданного момента времени) вызывает срабатывание `action`-функции, которую вы указали в качестве действия для этого триггера.

Временные триггеры, то есть те, которые срабатывают на заданное время, проверяются периодически на каждом шаге симуляции. Сработавший триггер удаляется из списка.

Позиционные триггеры проверяются по наступлении события "изменение состояния траектории". То есть когда где-либо на карте происходит либо занятие, либо освобождение траектории. Данные триггеры могут удаляться после срабатывания, а могут оставаться в системе и срабатывать снова и снова. Существует доступный разработчику сценария механизм управления временем жизни триггера.

4.2. Создание собственной `action`-функции

В [приложении 2](#) приведен список готовых `action`-функций, которые можно указывать в качестве действий для триггеров. Однако, этого функционала может не хватить, поэтому полезно научиться создавать свои `action`-функции.

Итак, `action`-функция - это функция на языке Lua. И выглядит она так

```
function my_action()

    -- Здесь пишем свою произвольную логику

end
```

Здесь:

- function - ключевое слово Lua, означающее начало описания функции
- my_action - имя функции, оно произвольное, но удовлетворяет ряду требований: латиница, не должно начинаться с цифры, не должно содержать пробелов, арифметических знаков, слешей, кавычек, скобок.
- () - в круглых скобках - список параметров, передаваемых в функцию. В данном примере нет параметров.
- end - ключевое слово, обозначающее конец тела функции.

Внутри функции можно описать произвольные действия, например вызвать системную функцию ([приложение 1](#)) построения поездного маршрута

```
function my_action()

    buildTrainRoute("track_a_p2b", "track_a-b_nd-22", 1)

end
```

Теперь нашу action-функцию мы можем передать в триггер, например во временной

```
setTimeTrigger("17:30:10", my_action)
```

Полный текст сценария, содержащего эти действия, таков

```
-- Задаем время начала игры
setTime("17:30")

-- Описываем поезд игрока
my_train = TrainData.new()
my_train.name = "ВЛ60пк"
my_train.config = "v160pk-1543"
my_train.traj = "track_a_p2b"
my_train.coord = 1690.0
my_train.dir = 1

-- Устанавливаем поезд игрока
setTrain(my_train)

-- Пользовательская action-функция
function my_action()

    buildTrainRoute("track_a_p2b", "track_a-b_nd-22", 1)

end
```

```
-- Ставим временной триггер с пользовательским action
setTimeTrigger("17:30:10", my_action)
```

То же самое действие, но с использованием стандартной action-функции `actionBuildTrainRoute("track_a_p1a", "track_a-b_nd-22", 1)` мы делали в [главе 2](#), что гораздо быстрее и проще. В чем разница?

Разница в том, что в нашей функции мы можем определить произвольные действия, например построить не один, а два маршрута

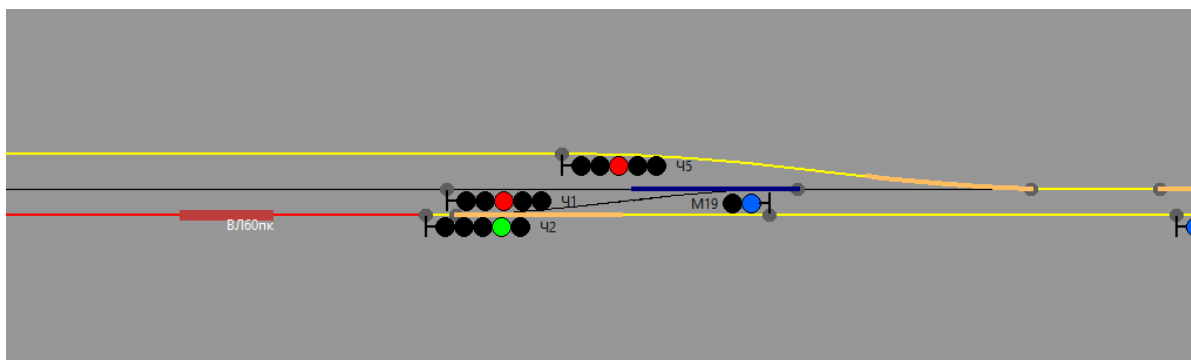
```
function my_action()

    -- Строим четный маршрут отправления
    buildTrainRoute("track_a_p2b", "track_a-b_nd-22", 1)

    -- Строим нечетный маршрут приема
    buildTrainRoute("track_a-b_n-1", "track_a_p5b", -1)

end
```

Видоизменив функцию, мы получаем следующую картину



Логика работы нашей action-функций может быть произвольной. К сожалению, для полного овладения всеми возможностями, которые предоставляет данный механизм, придется погрузиться в изучение языка Lua. Но некоторые вещи мы все же разберем здесь.

4.3. Передача параметров в action-функцию

В предыдущем примере мы сконструировали action-функции без параметров, а в вызываемые внутри системные функции передали конкретные имена траекторий и направление.

Однако, когда создается функция, обычно это подразумевает некоторые универсальные операции, выполняемые многократно.

Рассмотрим такую задачу - поставить временной триггер на построение сквозного пропуска в двух направлениях для произвольной станции. Причем action-функция будет одна единственная, а триггеров мы поставим два - один для станции В, другой для станции С.

Для сквозного пропуска нам надо знать имена участков приближения и удаления для каждой станции в каждом направлении. Это мы, разумеется, выясним, но прежде напомним универсальную action-функцию, строящую маршруты в двух направлениях.

```
-- Построение двух сквозных маршрутов пропуска
function dual_approach(traj_CHP1, traj_CHU1, traj_NP1, traj_NU1)
```

```

-- Объявляем функцию БЕЗ параметров
function action()

    -- Строим маршрут от траектории traj_CHP1 до traj_CNU1 в направлении 1
    buildTrainRoute(traj_CHP1, traj_CNU1, 1)
    -- Строим маршрут от траектории traj_NP1 до traj_NU1 в направлении -1
    buildTrainRoute(traj_NP1, traj_NU1, -1)

end

-- Возвращаем функцию
return action

end

```

Функция `dual_approach` будет строить два встречных сквозных маршрута по станции, и она имеет четыре параметра. Параметры функции указываются в скобках, через запятую. Имена параметров подчиняются тем же правилам, что и имена функций. Смысл данных параметров следующий

- `traj_CHP1` - первый четный участок приближения (имя траектории)
- `traj_CNU1` - первый четный участок удаления
- `traj_NP1` - первый нечетный участок приближения
- `traj_NU1` - первый нечетный участок удаления

Однако, видно, что внутри функции `dual_approach` у нас описана еще одна функция, без параметров. Дело в том, что движок игры, для универсальности организации таких функций, полагает их функциями без параметров. Поэтому нам нужна внешняя функция, с параметрами - именами участков приближения и удаления, и внутренняя функция, без параметров, которая вызовет системные функции построения маршрута, передав им параметры внешней функции.

Обратите внимание на эту строку

```

-- Возвращаем функцию
return action

```

Ключевое слово `return` означает, что функция возвращает некоторое значение, результат своей работы, который может быть использован в дальнейшем. В данном случае мы возвращаем *функцию*. Внутри функции `dual_approach` мы конструируем `action`-функцию, используем для её логики переданные параметры, а потом возвращаем эту функцию как результат работы.

А далее мы пишем вот так

```

setTimeTrigger("17:30:20", dual_approach("track_a-b_2-ch", "track_b-c_nd-22",
"track_b-c_n-1", "track_a-b_21-chd"))

```

Ставим триггер на время 17:30:20, а в качестве `action`-функции передаем результат работы функции `dual_approach`, которая создает `action`-функцию для триггера с нужными нам параметрами.

В данном случае в 17:30:20 будут построены два сквозных маршрута по станции В. Если мы хотим проделать то же самое и для станции С, поставим еще один триггер

```
setTimeTrigger("17:30:20", dual_approach("track_b-c_2-ch", "track_c_m3-nd",  
"track_c_m1-n", "track_b-c_21-chd"))
```

Здесь мы изменили только имена участков приближения и удаления, остальная логика осталась неизменной.

Такой механизм, когда мы конструируем функцию "на ходу", передаем её для использования другой функции, и при этом она может выполняться не прямо сейчас, а гораздо позже, по наступлению определенного события, носит название "анонимная функция", или лямбда-функция. Такой подход широко используется в скриптовом языке Lua, движок сценариев RRS активно его эксплуатирует.

4.4. Триггеры, устанавливающие триггеры, устанавливающие триггеры...

С помощью action-функций можно добиться того, что, например, установленный нами триггер на одно событие может сам установить триггер уже на другое событие. Это позволяет создать в сценарии цепочку взаимосвязанных друг с другом действий, не обязательно четко привязанных к конкретному времени.

Например, рассмотрим такую ситуацию. На перегоне, по направлению к станции В, один за другим следуют два поезда. Первый поезд мы принимаем на боковой путь, а второму - организуем сквозной пропуск. Затем, когда второй поезд проследует по станции В, нам надо отправить первый поезд по удалению за вторым. То есть мы ставим первый поезд под обгон вторым, а все операции по построению маршрутов хотим возложить на сценарий.

Первым вариантом решения такой задачи могло бы быть создание временных триггеров, которые отработают последовательно на построение маршрутов. Но кто дает гарантию, что мы угадаем моменты времени, когда следует строить тот или другой маршрут? Разумеется, поезда должны следовать по графику, и определенные временные рамки всех перечисленных операций у нас имеются. Но мы не можем и не должны полагаться на это! В реальной жизни, дежурный по станции прежде всего готовит маршруты поезда, исходя из свободности и занятости участков станции, а соблюдение графика складывается в целом из его грамотных действий и грамотной работы машинистов поездов.

Поэтому мы воспользуемся позиционными триггерами. Пока попробуем обойтись стандартными триггерами ([приложение 3](#)) и собственными action-функциями, там, где это действительно требуется.

Итак, создадим два пассажирских поезда, для ускорения демонстрации примера расположим их на перегоне, на достаточном удалении друг от друга, так, чтобы их разделяло два блок-участка. Первый поезд поставим перед вторым участком приближения к станции В с четного направления.

```
-- Устанавливаем поезд 1  
train1 = TrainData.new()  
train1.name = "1"  
train1.config = "v160pk-1543-T65_17"  
train1.traj = "track_a-b_6-4"  
train1.coord = 1900.0  
train1.dir = 1  
  
setTrain(train1)
```

```
-- Устанавливаем поезд 3
train3 = TrainData.new()
train3.name = "3"
train3.config = "v160pk-1543-T65_17"
train3.traj = "track_a-b_12-10"
train3.coord = 1900.0
train3.dir = 1

setTrain(train3)
```

Первый поезд мы принимаем на 4 путь. Маршрут ему надо построить, когда он вступит на второй участок приближения к станции В с четного направления. Имя траектории, которая является этим участком, мы можем определить из карты, но для удобства запишем его в переменную

```
-- Второй участок приближения к станции В
local stB_CHP2 = "track_a-b_4-2"
```

Это имя мы будем использовать несколько раз, поэтому его удобно записать в локальную переменную с именем stB_CHP2. Ключевое слово local означает, что данная переменная видна только внутри скрипта main.lua.

Заведем переменные для других траекторий, занятие и освобождение которых мы будем отслеживать. Они же будут участвовать в построении маршрутов. То есть опять-таки удобно объявить эти имена как переменные

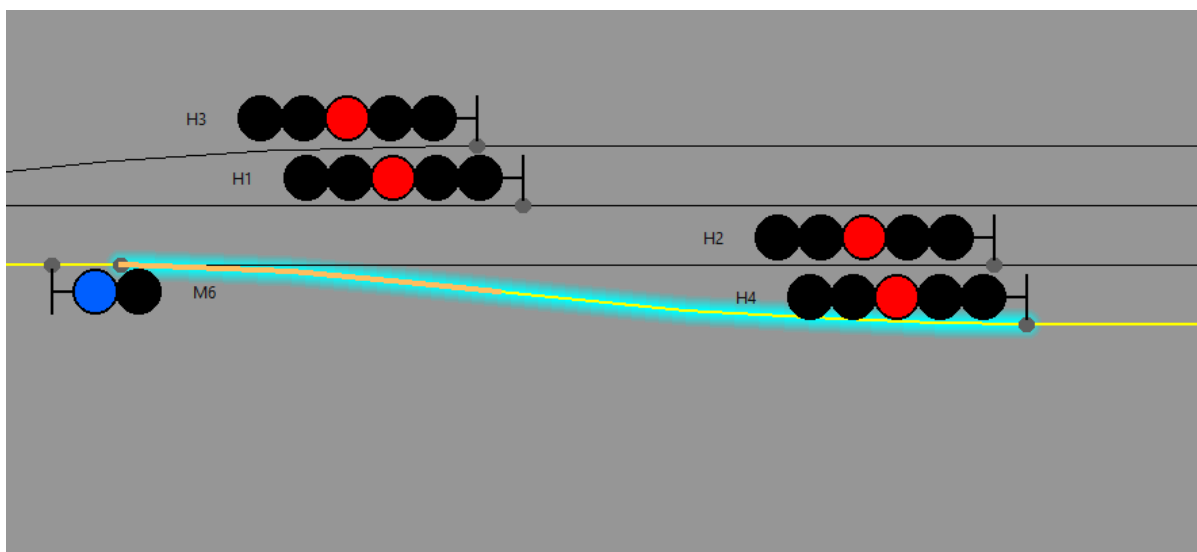
```
-- Первый участок удаления от станции В
local stB_CHU1 = "track_b-c_nd-22"
-- Второй участок удаления от станции В
local stB_CHU2 = "track_b-c_22-20"
-- 4 путь станции В
local stB_p4 = "track_b_p4"
-- Стрелочный участок от стрелки 12 до 4 пути станции В
local sw12_n4 = "track_b_12-p4"
```

Построим маршрут приема для поезда 1, по факту его вступления на второй участок приближения

```
-- Строим маршрут приема поезду 1 на 4 путь станции В
setOnTrajBusyTrigger(stB_CHP2, actionBuildTrainRoute(stB_CHP2, stB_p4,
train1.dir))
```

Используем стандартный триггер, срабатывающий по занятию указанной траектории, и стандартный action построения поездного маршрута, от второго участка приближения, до 4 пути. Вместо конкретных названий траекторий используем *переменные*, в которых хранятся данные имена.

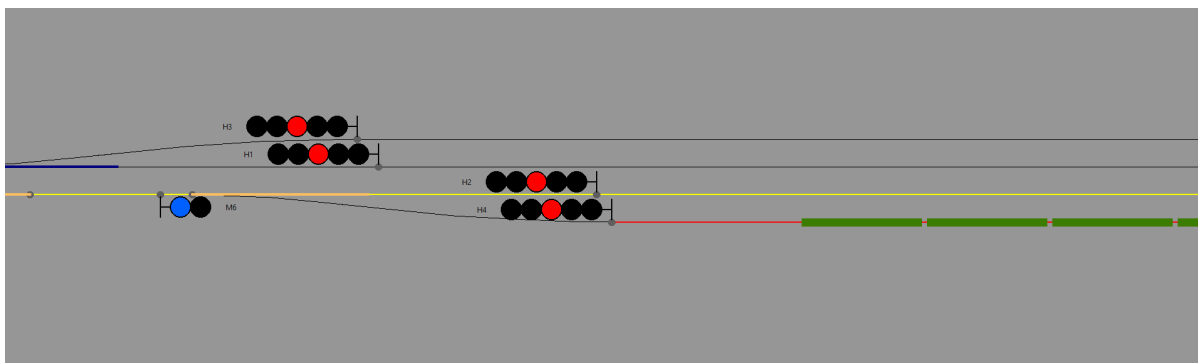
Когда нам следует построить маршрут для поезда 3? Тогда, когда это станет возможным, а именно когда поезд 1 не будет уже занимать необходимые для построения этого маршрута стрелки. Например, когда поезд 1 полностью заедет на 4 путь, а значит он освободит стрелочный участок, примыкающий к этому пути, то есть траекторию, показанную на рисунке



Имя этой траектории мы записали в переменную sw12_H4. По факту её освобождения мы можем построить маршрут пропуска поезду 3, например так

```
setOnTrajFreeTrigger(sw12_H4, actionBuildTrainRoute(stB_CHP2, stB_CHU1,
train3.dir))
```

При текущем расположении поездов это безусловно сработает - как только поезд 1 освободит стрелочный участок, заехав на 4 путь, маршрут поезду 3 будет построен.



Но что, если дистанция между поездами будет другой? Если поезда будут ближе друг к другу, вплоть до ситуации, когда они занимают два подряд блок участка, работа сигнализации на перегоне не даст им сблизиться на дистанцию менее длины одного блок участка. Но все равно, чтобы избежать ситуации блокировки построения маршрута приема/пропуска, лучше строить его не от второго, а от первого участка приближения

```
-- Первый участок приближения к станции В
local stB_CHP1 = "track_a-b_2-ch"
```

и для маршрутов перепишем триггеры

```
-- Строим маршрут приема поезду 1 на 4 путь станции В
setOnTrajBusyTrigger(stB_CHP2, actionBuildTrainRoute(stB_CHP1, stB_p4,
train1.dir))

-- Строим маршрут пропуска поезду 3
setOnTrajFreeTrigger(sw12_H4, actionBuildTrainRoute(stB_CHP1, stB_CHU1,
train3.dir))
```

Если расстояние между поездами будет более значительным, может пройти достаточно большое время, прежде чем потребуется маршрут пропуска для поезда 3. И если мы построим маршрут сразу, то главный путь и все сопряженные с ним участки станции будут заняты до тех пор, пока поезд 3 не будет пропущен по маршруту. Поэтому лучше, в данной ситуации, строить маршрут поезду 3 так же, как и для поезда 1 - по факту занятия им второго участка приближения. А для этого нам нужно установить триггер на построение этого маршрута уже после того, как поезд 1 освободит стрелочный участок. Напишем для этого собственную action-функцию

```
-- Действия, выполняемые при освобождении поездом 1 стрелочного участка 12 - путь 4
function set_appr_train3()

    -- Строим маршрут пропуска поезду 3
    setOnTrajBusyTrigger(stB_CHP2, actionBuildTrainRoute(stB_CHP1, stB_CHU1,
train3.dir))

end
```

Эта функция выполняет операцию установки нового позиционного триггера на занятие второго участка приближения, но уже с построением сквозного маршрута для поезда 3. Эту функцию мы подставим в качестве action для триггера на освобождение стрелочного участка перед 4 путем

```
setOnTrajFreeTrigger(sw12_n4, set_appr_train3)
```

Теперь при освобождении стрелочного участка маршрута не будет строиться маршрут пропуска, а будет установлен триггер, который сработает при занятии поездом 3 второго участка приближения и построит ему маршрут.

И наконец, чтобы завершить обгон, нам надо построить маршрут поезду 1. Мы сможем это сделать, когда поезд 3 покинет хотя бы первый, а лучше - второй участок удаления. Делаем эту операцию путем установки позиционного триггера на освобождение второго участка удаления

```
-- Строим маршрут отправления поезду 1 с 4 пути, когда освободится второй участок
удаления
setOnTrajFreeTrigger(stB_CHU2, actionBuildTrainRoute(stB_p4, stB_CHU1,
train1.dir))
```

И на этом примере продемонстрируем еще один временной триггер, устанавливаемый функцией `setPostEventTimeTrigger()` (см. [приложение 1](#))

Этот триггер срабатывает по времени, *через промежуток времени, прошедший с момента его установки.*

Например, мы хотим построить маршрут отправления поезду 1 не сразу после того, как поезд 3 освободит второй участок удаления, а через 1 минуту 20 секунд после этого. Для этого создадим свою action-функцию

```
-- Действия, выполняемые после освобождения поездом 3 второго участка удаления
function set_dep_train1()

    -- Строим маршрут отправления поезду 1 через 1 минуту 20 секунд после
    освобождения ЧУ2
    setPostEventTimeTrigger("+00:01:20", actionBuildTrainRoute(stB_p4, stB_CHU1,
    train1.dir))

end
```

и устанавливаем её в качестве действия на триггер освобождения второго участка удаления

```
setOnTrajFreeTrigger(stB_CHU2, set_dep_train1)
```

Таким образом, при использовании собственных action-функций можно комбинировать различные триггеры, устанавливая их в том числе и каскадно, создавая достаточно сложные цепочки событий и действий по управлению движением из сценария.

4.5. Внутренняя организация позиционного триггера.

Функция триггера. Время жизни позиционного триггера.

Те позиционные триггеры, о которых мы до сих пор говорили, описанные в [приложении 3](#), являются стандартными и обрабатывают типовые ситуации. Эти триггеры работают поверх системного слоя симулятора, реализованы на языке Lua, а их реализацию можно увидеть в файле [modules/lua/triggers fabric.lua](#). В него не возбраняется заглянуть и увидеть, что реализация достаточно сложная. Возьмем для примера функцию setOnTrajFreeTrigger()

```
-----
-- Занятие траектории произвольным поездом
-----
function setOnTrajBusyTrigger(name_traj, action)

    function on_busy_trigger_func(train_name, traj_name, is_busy)

        if is_busy and traj_name == name_traj then

            action()

            return TRIG_DELETE
        end

        return TRIG_SAFE
    end

    setTrigger(on_busy_trigger_func)
end
```

Напомню, что setOnTrajFreeTrigger - это функция, которая устанавливает триггер (событие), срабатывающий при занятии заданной траектории произвольным поездом. В эту функцию передается два параметра: имя траектории (name_traj), занятие которой проверяется, и функция, которая должна при этом выполняться (action).

Внутри реализации описана еще одна функция

```
function on_busy_trigger_func(train_name, traj_name, is_busy)

    if is_busy and traj_name == name_traj then

        action()

        return TRIG_DELETE
    end

    return TRIG_SAFE
end
```

Это - так называемая *функция позиционного триггера*. Она может иметь любое разрешенное имя, но имеет четкий список параметров, порядок которых нельзя нарушать

- train_name - имя поезда, занявшего или освободившего траекторию
- traj_name - имя траектории, которая была занята или освобождена
- is_busy - состояние занятости траектории: true - занята, false - свободна

Каждый раз, когда в процессе игры происходит изменение состояния какой-либо траектории, движок симулятора обращается к списку позиционных триггеров, за каждым из которых закреплена такая вот функция. Симулятор вызывает эти функции по очереди и в *каждую из них* передает имя поезда, изменившего состояние траектории, имя траектории, которая поменяла свое состояние, и то состояние (is_busy), которая она получила.

То есть сам симулятор вызывает эту функцию и передает туда перечисленную информацию. И дальше дело за разработчиком, как использовать её.

В данном примере стандартного триггера происходит проверка **одновременного выполнения** двух условий

- траектория была занята (is_busy - истинно)
- имя занятой траектории совпадает с той, что мы передали в функцию установки триггера (traj_name == name_traj)

В случае, если оба условия выполнены, значит наступило то событие, которое мы ожидаем, и тогда выполняется функция action(), которая, как мы видели выше, может содержать произвольную последовательность действий. После этого следует выход из функции, с передачей симулятору кода выхода TRIG_DELETE

```
return TRIG_DELETE
```

Симулятор, приняв код выхода TRIG_DELETE, понимает, что данный триггер больше не нужен, и удаляет его из списка обрабатываемых триггеров.

Если хотя бы одно из этих условий неверно, функция action() не выполняется. И мы выходим из функции, передав симулятору другой код выхода - TRIG_SAFE, что указывает на то, что триггер следует сохранить в списке. В этом случае он будет проверяться на срабатывание снова и снова, при каждом изменении состояния траекторий в игре.

Операция возврата кода выхода - обязательна при реализации функции триггера. Симулятор ожидает получить указание о том, что делать с триггером, после того как будет вызвана и отработает данная функция. Таким способом, логика внутри триггерной функции может управлять временем жизни позиционного триггера.

В конце функции `setOnTrajBusyTrigger` содержится вызов

```
setTrigger(on_busy_trigger_func)
```

`setTrigger` - это системная функция (см. [приложение 1](#)) установки позиционного триггера. В качестве единственного параметра она принимает функцию триггера, в данном случае это функция `on_busy_trigger_func`, определенная выше. Это явное указание симулятору добавить данный триггер в список контролируемых позиционных триггеров.

ВАЖНО!!! Функция позиционного триггера имеет строго определенный формат!

```
function pos_trig_func(train_name, traj_name, is_busy)

    -- Здесь пишем произвольную логику, которая может использовать
    -- параметры train_name, traj_name, is_busy, получаемые от симулятора

    return TRIG_SAFE

end
```

Эта функция обязательно имеет три параметра: имя поезда, имя траектории, состояние, которое приняла траектория. Именно в этом порядке!

Эта функция обязательно должна возвращать симулятору код выхода!

Для установки позиционного триггера, обрабатываемого через данную функцию, обязательно выполняем вызов

```
setTrigger(pos_trig_func)
```

Это - еще один инструмент расширения функциональности движка сценариев. Да, предусмотрены стандартные триггеры на занятие/освобождение траекторий (см. [приложение 3](#)). Возможно, по результатам опробования системы сценариев пользователями, данный список будет расширен. Однако, все возможные ситуации предусмотреть нереально.

Ниже я приведу пример того, как это можно использовать на практике. Но предварительно мы рассмотрим один небольшой вопрос.

4.6. Некоторые сервисные функции, предоставляемые игрой

Допустим, наша логика предусматривает необходимость проверки состояния какого-либо участка станции или перегона, и от этого будет зависеть то, как будет работать сценарий дальше. Для получения состояния траектории в любой момент предусмотрена системная функция `getTrajState` (см. [приложение 1](#)). Эта функция принимает на вход имя траектории и возвращает состояние этой траектории в данный момент.

Пример использования

```
local traj_state = getTrajState("track_a_p1a")
```

Данная конструкция проверит состояние траектории с именем "track_a_p1a" и запишет его в переменную traj_state. Переменная traj_state является *структурой* и содержит два параметра

- is_busy - признак того, что траектория занята (true - занято, false - свободно)
- in_route - признак того, что данная траектория включена в уже построенный маршрут

Дальнейшее использование результата работы этой функции предусматривает применение условий, например

```
-- Проверяем занятость траектории
if traj_state.is_busy then

    -- Действия, которые мы предпримем, если траектория занята

end

-- Проверяем включение траектории в маршрут
if traj_state.in_route then

    -- Действия, которые мы предпримем, если траектория включена в маршрут

end
```

Еще одна функция, которая может быть полезна - getNextTraj. Она принимает два параметра - имя траектории и направление (1 или -1), и возвращает имя траектории, которая следует за данной траекторией в указанном направлении. Если траектория в заданном направлении оканчивается стрелкой, то вернется имя той траектории, на которую переключена стрелка в данный момент.

Пример использования:

```
local next_traj = getNextTraj("track_a_p1a", 1)
```

в переменную next_traj будет записано имя траектории, которая следует за траекторией "track_a_p1a" в направлении 1 ("туда").

Следующая функция может быть полезна при отладке сценариев. Эта функция выводит произвольное текстовое сообщения в лог симулятора logs/journal.log

```
logMessage("hello from RRS scenario system!")
```

Это может помочь разработчику понять, какие части его сценария выполняются и в какое время. Данные сообщения содержат строку "LUA DEBUG:", которая поможет вам отделить их от достаточно большого количества информации, выводимой в лог при работе игры.

4.7. Практическое применение функции позиционного триггера

Представим себе ситуацию, когда мы ставим триггер на занятие второго участка приближения, рассчитывая построить маршрут пропуска для поезда. Хорошо, если все необходимые участки станции будут при этом свободны. Но может ли оказаться иначе? Разумеется. Например, горловина станции может быть занята другим зазевавшимся поездом. Или участки, через которые будет пролегать маршрут пропуска, будут включены в другой маршрут, например маршрут отправления другого поезда.

Разумеется, если движение в сценарии организуется по графику, такие ситуации могут и не возникнуть, но гарантировать этого нельзя. Покажем, как средствами сценария можно подобную историю грамотно обработать.

Подготовим новый сценарий для всего этого безобразия

```
-- Устанавливаем поезд 102
train102 = TrainData.new()
train102.name = "102"
train102.config = "vl60pk-1543-T65_17"
train102.traj = "track_b_p4"
train102.coord = 1250.0
train102.dir = 1
train102.auto = true

setTrain(train102)

-- Устанавливаем поезд 104
train104 = TrainData.new()
train104.name = "104"
train104.config = "vl60pk-1543-T65_17"
train104.traj = "track_a-b_6-4"
train104.coord = 1900.0
train104.dir = 1
train104.auto = true

setTrain(train104)

-- Второй участок приближения к станции В
local stB_CHP2 = "track_a-b_4-2"
-- Первый участок приближения к станции В
local stB_CHP1 = "track_a-b_2-ch"
-- Первый участок удаления от станции В
local stB_CHU1 = "track_b-c_nd-22"
```

Поезд 104 находится рядом со вторым участком приближения. Но мы бездумно построим маршрут отправления поезду 102, который стоит на 4 пути станции

```
setTimeTrigger("+00:00:36", actionBuildTrainRoute(train102.traj, stB_CHU1,
train102.dir))
```

Пока поезд 104 доедет до второго приближения, маршрут уже будет построен. Соответственно горловина станции В будет занята, и сквозной маршрут этому поезду построить не выйдет. Однако, создав проблему, мы героически решим её с помощью такой функции для позиционного триггера

```
-- Специальная функция для позиционного триггера
function train104_appr(train_name, traj_name, is_busy)

    -- Если занята траектория stB_CHP2
    if is_busy and traj_name == stB_CHP2 then

        -- Проверяем состояние участка удаления
        local traj_state = getTrajState(stB_CHU1)

        -- Если он занят или включен в маршрут
        if traj_state.is_busy or traj_state.in_route then

            -- Ставим триггер на его освобождение с целью таки построить маршрут
            setOnTrajFreeTrigger(stB_CHU1, actionBuildTrainRoute(stB_CHP1,
stB_CHU1, train104.dir))

        else

            -- Если всё ок, строим маршрут
            buildTrainRoute(stB_CHP1, stB_CHU1, train104.dir)

        end

        -- В любом случае триггер удаляем, он отработал
        return TRIG_DELETE

    end

    -- Оставляем триггер, если он не сработал
    return TRIG_SAFE

end
```

Триггер срабатывает на условие занятия второго участка приближения

```
-- Если занята траектория stB_CHP2
if is_busy and traj_name == stB_CHP2 then
```

Однако, не спешим строить маршрут, для начала проверим состояние первого участка удаления

```
-- Проверяем состояние участка удаления
local traj_state = getTrajState(stB_CHU1)
```

Далее, если он или занят, или участвует в другом маршруте

```
if traj_state.is_busy or traj_state.in_route then
```

мы ставим триггер на его освобождение, и уже этот триггер построит для поезда 104 маршрут пропуска, когда поезд 102 уберется с первого удаления

```
-- Ставим триггер на его совобожение с целью таки построить маршрут
setOnTrajFreeTrigger(stB_CHU1, actionBuildTrainRoute(stB_CHP1,
stB_CHU1, train104.dir))
```

Если же вдруг участок удаления окажется свободен, просто построим маршрут пропуска

```
-- Если всё ок, строим маршрут
buildTrainRoute(stB_CHP1, stB_CHU1, train104.dir)
```

Отработавший триггер, независимо от занятости траектории удаления, мы приказываем убрать из системы

```
-- Влюбом случае триггер удаляем, он отработал
return TRIG_DELETE
```

Оставляем триггер только в том случае, если второй участок приближения еще не занимался

```
-- Оставляем триггер если он не сработал
return TRIG_SAFE
```

Не забываем установить нашу триггерную функцию в системе

```
setTrigger(train104_appr)
```

Надо сказать, что данный пример довольно синтетический, однако он иллюстрирует, что функция позиционного триггера может быть написана самостоятельно и содержать довольно развесистую логику.

5. График движения поезда

Система сценариев симулятора была бы не полной без такого важного аспекта, как возможность задавать график движения поезда. И в RRS имеется такая функциональность. График движения поезда в игре выполняет следующие функции:

1. Информировует игрока о маршруте следования поезда, с указанием станций, времени прибытия и отправления.
2. Отслеживает фактическое время прибытия и отправления поезда, с информированием игрока о соблюдении графика.
3. Обеспечивает автоматическое следования поезда по графику, если поезд находится в режиме "Автоведение" (параметр auto в [структуре TrainData](#))
4. Обеспечивает автоматическое построение маршрутов приема/пропуска/отправления поезда по заданному пути станции.

Рассмотрим пример создания графика движения для сценария на маршруте "Испытательный полигон 2", входящий в комплект поставки игры.

5.1 Создание графика движения

Графики движения всех поездов в данном сценарии располагаются в папке со сценарием. Для размещения графика движения необходимо создать в папке сценария каталог timetable.

Создадим сценарий, назвав его trains_134_135, и в главном файле main.lua напишем такую конфигурацию

main.lua

```
-- Время начала игры (важно!)
setTime("11:59")

-- Конфигурация поезда 134
train134 = TrainData.new()
train134.name = "134"
train134.config = "v160pk-1543-T65_17"
train134.traj = "track_a_p2b"
train134.coord = 1040.0
train134.dir = 1
train134.auto = true

setTrain(train134)

-- Конфигурация поезда 135
train135 = TrainData.new()
train135.name = "135"
train135.config = "v160pk-1543-T65_17"
train135.traj = "track_c_p2"
train135.coord = 30.0
train135.dir = -1
train135.auto = true

setTrain(train135)
```

В каталоге сценария создаем папку timetable - в ней будут храниться файлы графиков движения. Эти файлы имеют формат XML.

ВНИМАНИЕ!!! Имя файла должно совпадать с именем поезда!

Например, для поезда с именем "134" имя графика движения будет 134.xml, а для поезда с именем "135" - 135.xml

ВНИМАНИЕ!!! Файлы графиков движения должны сохраняться в кодировке UTF-8, принятой в симуляторе для всех текстовых данных! Используйте текстовый редактор с поддержкой сохранения файлов в данной кодировке.

Создадим в каталоге timetable файл 134.xml такого вида

timetable/134.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<Config>

  <Station>
    <Name>Станция А</Name>
    <DepartureTime>12:00</DepartureTime>
    <TargetTraj>track_a_p2b</TargetTraj>
    <TargetCoord>1040.0</TargetCoord>
    <RemovalTraj>track_a-b_nd-22</RemovalTraj>
  </Station>

  <Station>
    <Name>Станция В</Name>
    <ArrivalTime>12:20</ArrivalTime>
    <DepartureTime>12:25</DepartureTime>
    <TargetTraj>track_b_p4</TargetTraj>
    <TargetCoord>1250.0</TargetCoord>
    <ApproachTraj>track_a-b_4-2</ApproachTraj>
    <RemovalTraj>track_b-c_nd-22</RemovalTraj>
  </Station>

  <Station>
    <Name>Станция С</Name>
    <ArrivalTime>12:45</ArrivalTime>
    <TargetTraj>track_c_p3</TargetTraj>
    <TargetCoord>1250.0</TargetCoord>
    <ApproachTraj>track_b-c_4-2</ApproachTraj>
  </Station>

</Config>
```

Разберем этот файл подробнее. Он должен начинаться с шапки

```
<?xml version="1.0" encoding="UTF-8"?>
```

стандартной для всех XML-конфигов симулятора, и открывающего тега

```
<Config>
```

В конце файла обязан присутствовать закрывающий тег

```
</Config>
```

Между тегами Config располагается описание графика движения, который состоит из секций с именем Station. Каждая секция описывает станцию, или произвольный пункт на маршруте, через который должен проследовать поезд в заданное время.

ВНИМАНИЕ!!! Секции Station идут в порядке их расположения по ходу следования поезда!

Разберем подробно секцию, которая описывает станцию B.

```
<Station>
  <Name>Станция B</Name>
  <ArrivalTime>12:20</ArrivalTime>
  <DepartureTime>12:25</DepartureTime>
  <TargetTraj>track_b_p4</TargetTraj>
  <TargetCoord>1250.0</TargetCoord>
  <ApproachTraj>track_a-b_4-2</ApproachTraj>
  <RemovalTraj>track_b-c_nd-22</RemovalTraj>
</Station>
```

Эта секция включает ряд параметров

- Name - имя станции. Произвольное название, отображаемое в интерфейсе игры
- ArrivalTime - время прибытия на станцию по графику в формате hh:mm
- DepartureTime - время отправления со станции по графику.

Обращаю внимание на то, что если время прибытия совпадает с временем отправления, то поезд не будет останавливаться на станции, а проследует её без остановки. Однако, в таком случае **все равно должны быть явно указаны и время прибытия, и время отправления!**

Для станции отправления, самой первой станции в графике, допускается не указывать время прибытия - по сценарию поезд уже будет находиться на станции, и симулятор считает его прибывшим в момент времени начала игры.

Для конечной станции, самой последней в графике, допускается не указывать время отправления. Если не указать время отправления, поезд останется на станции до конца игры. Перейдем к другим параметрам

- TargetTraj - траектория, на которой поезд должен остановиться по графику или которую он должен проследовать. В данном случае это путь 4 станции B ("track_b_p4").
- TargetCoord - координата точки остановки поезда в пределах траектории. Она находится в пределах от 0 до длины траектории. Изменяя эту координату, можно добиться, например, остановки поезда в пределах платформы. При проследовании без остановки, поезд должен двигаться по данной траектории. Пересечение точки с указанной координатой фиксирует фактическое время отправления поезда со станции.
- ApproachTraj - имя траектории, которая является участком приближения к станции. Если указан данный параметр, то при занятии поездом этой траектории будет автоматически построен маршрут приема поезда на путь, заданный параметром TargetTraj.
- RemovalTraj - имя траектории, которая является участком удаления от станции. Если указан этот параметр, то будет автоматически построен маршрут отправления поезда с пути TargetTraj через траекторию RemovalTraj.

ВНИМАНИЕ!!! Если указаны оба параметра: ApproachTraj и RemovalTraj - то при приближении поезда к станции будет автоматически построен маршрут на сквозной пропуск через путь TargetTraj. Поэтому оба параметра рекомендуется задействовать именно при сквозном пропуске без стоянки либо при небольшом времени стоянки. Если в течение времени стоянки предполагается построение других маршрутов через RemovalTraj, то этот параметр лучше не указывать, дабы не занимать СЦБ станции построенным заранее маршрутом. В этом случае для отправления поезда с данной станции лучше прописать в сценарии временной триггер.

Кроме того, существует ряд параметров, необязательных для использования.

- IsRightPlatform - наличие платформы справа от целевого пути. Принимает значения 1 или 0, обозначающие, соответственно, наличие и отсутствие платформы справа. Указывает модулю автоведения на необходимость открыть правые двери вагонов (реализовано, например, в дизель-поезде РА-3).
- IsLeftPlatform - наличие платформы слева от целевого пути. Принимает значения 1 или 0, обозначающие, соответственно, наличие и отсутствие платформы справа. Указывает модулю автоведения на необходимость открыть левые двери вагонов (реализовано в дизель-поезде РА-3).
- IsVisible - станция видна в виджете графика поезда. Этот параметр можно не указывать, тогда станция будет отображаться в виджете графика в симуляторе. Если станцию надо скрыть, можно в секции Station определить этот параметр как 0

```
-- Станция не видна в виджете графика
<Isvisible>0</Isvisible>
```

Такие скрытые точки графика можно использовать, например, для того, чтобы запланировать опоздание поезда. Для этого путевую точку можно разместить на перегоне, указав время проследования этой точки или вообще остановки на заданное время. Например, в описанном выше графике можно задать такую путевую точку между станциями А и В

```
<Station>
  <Name>Остановка на перегоне между А и В</Name>
  <ArrivalTime>12:10</ArrivalTime>
  <DepartureTime>12:18</DepartureTime>
  <TargetTraj>track_a-b_8-6</TargetTraj>
  <TargetCoord>900.0</TargetCoord>
  <Isvisible>0</Isvisible>
</Station>
```

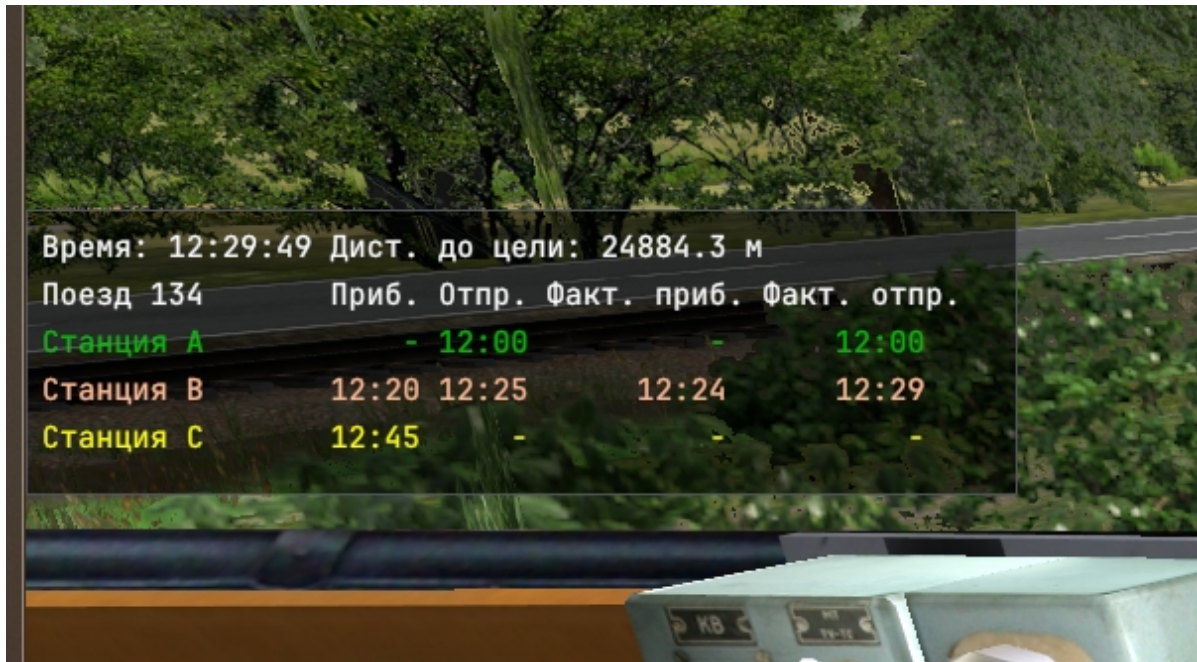
В этом случае поезд, движущийся автоматически, будет ехать так, что прибудет в точку, расположенную через 900 метров после сигнальной точки 8 на перегоне между станциями в 12:10, остановится там и, простояв 8 минут, отправится. При этом он почти наверняка опоздает с прибытием на станцию В по графику.

При этом данная точка не будет отображаться на графике движения.

Для справки можно использовать краткое описание формата графика в [приложении 5](#)

5.2 Виджет отображения графика в интерфейсе симулятора

Если игра происходит по сценарию и для данного поезда задан график движения, его можно отобразить на экране путем нажатия клавиши F7. Это выглядит так



Виджет отображает текущее время, расстояние до ближайшей по ходу движения станции, а также перечень станций, с указанием времени прибытия и времени отправления по графику, а также фактического времени прибытия и фактического времени отправления.

Проследованные поездом станции обозначаются цветом: зеленым - в случае проследования по графику (+/- 2 минуты), оранжевым - в случае отклонения от графика на 3 минуты и более.

Желтой строкой обозначается станция-цель, к которой в данный момент движется поезд.

5.3 Загрузка графика в процессе игры

В [главе 3](#) мы рассматривали ситуацию, когда имя поезда может измениться. Это может произойти после сцепления локомотива с составом или после отцепки его от состава - появляется поезд, не существовавший в игре при старте.

Хотя имя поезда может быть любым, мы подразумеваем под именем поезда его номер, а номер всегда соответствует графику. Что будет, если мы переименуем поезд?

Симулятор сбросит старый график и попытается найти новый график, в соответствии с новым именем (номером) поезда. Если таковой график есть в каталоге timetable сценария, то он будет успешно загружен.

Напомню, что переименование поезда возможно тремя способами: из кабины локомотива (F8), из интерфейса карты (правой кнопкой по имени поезда -> "Переименовать"), а также по триггеру в сценарии с помощью action-функции `actionRenameTrain()`.

Пример переименования поезда в процессе исполнения сценария мы не рассматривали. Рассмотрим в этот раз. Пусть поезд едет от станции А до станции В по графику поезда 134, который мы привели выше, а на станции В меняет номер на 136. Создадим еще один график для поезда 136, в котором, для простоты, время прибытия на станцию В будет таким же, как и в графике 134.

timetable/136.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<Config>
  <Station>
    <Name>Станция А</Name>
    <DepartureTime>12:00</DepartureTime>
    <TargetTraj>track_a_p2b</TargetTraj>
    <TargetCoord>1040.0</TargetCoord>
    <RemovalTraj>track_a-b_nd-22</RemovalTraj>
  </Station>

  <Station>
    <Name>Станция В</Name>
    <ArrivalTime>12:18</ArrivalTime>
    <DepartureTime>12:28</DepartureTime>
    <TargetTraj>track_b_p4</TargetTraj>
    <TargetCoord>1250.0</TargetCoord>
    <ApproachTraj>track_a-b_4-2</ApproachTraj>
    <RemovalTraj>track_b-c_nd-22</RemovalTraj>
  </Station>

  <Station>
    <Name>Станция С</Name>
    <ArrivalTime>12:40</ArrivalTime>
    <TargetTraj>track_c_p3</TargetTraj>
    <TargetCoord>1250.0</TargetCoord>
    <ApproachTraj>track_b-c_4-2</ApproachTraj>
  </Station>
</Config>
```

После станции В этот график отличается от графика 134. Сменим имя поезда на 136 в момент его прибытия на станцию В. Для этого напишем такой сценарий

main.lua

```
-- Задаем время
setTime("11:59")

-- Задаем поезд 134
train134 = TrainData.new()
train134.name = "134"
train134.config = "v160pk-1543-T65_17"
train134.traj = "track_a_p2b"
train134.coord = 1040.0
train134.dir = 1
train134.auto = true

setTrain(train134)

-- Меняем имя поезда с 134 на 136 в момент прибытия на станцию В
setTimeTrigger("12:20", actionRenameTrain(train134.name, "136"))
```

Функция `actionRenameTrain()` может применяться и в других типах триггеров [аналогично тому, как это описано в главе 4](#).

5.4 Создание попутных и встречных поездов

Для организации движения попутных и встречных поездов в сценарии (кроме поездов игроков), предусмотрен режим автоматического управления движением. Все поезда сценария моделируются физически, и, соответственно, для формирования встречных и попутных поездов используются те же принципы, что и для создания поезда игрока. Поезда можно формировать из всего представленного в игре подвижного состава.

Для использования режима автоматического управления в сценарии следует указать параметр `auto` для соответствующего поезда

```
-- Включение режима "Автоведение"  
train134.auto = true
```

При запуске игры данный поезд автоматически приводится в рабочее состояние и переходит в режим автоматического ведения. При этом

- Рассчитывается скорость движения для соблюдения графика, если график задан.
- Определяется текущее ограничение скорости, с учетом снижения скорости при следующем более строгом ограничении
- Определяется скорость движения, с учетом сигнала светофора.
- Из вышеперечисленных скоростей выбирается наименьшая.
- Выполняется автоматическое управление тягой и торможением.

В любой момент режим автоведения может быть выключен нажатием клавиш `Alt + F`. Этот режим может быть включен в любой момент повторным нажатием клавиш `Alt + F`.

Таким образом, все поезда, существующие в данной игровой сессии, имеют возможность управляться автоматически или в ручном режиме (игроком). Понятия трафик, в том смысле, который вкладывается в данный термин в других железнодорожных симуляторах, в RRS нет - все поезда, существующие в игре, абсолютно равноценны, и к ним применяются все имеющиеся игровые механики.

Приложение 1. Перечень системных функций

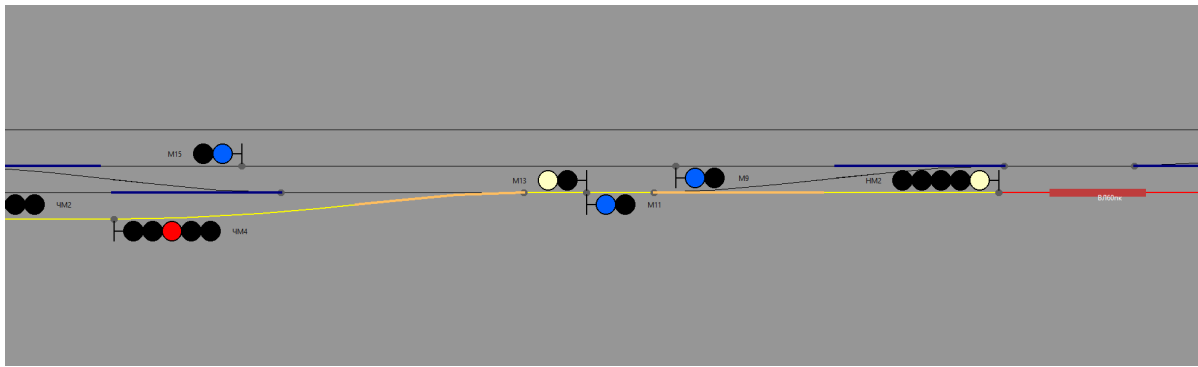
Функция	Параметры	Описание
setTime	"hh:mm:ss"	Установить время начала игры (допускается опустить секунды)
setDate	"dd.mm.yyyy"	Установить дату начала игры
setDateTime	"dd.mm.yyyy hh:mm:ss"	Установить дату и время начала игры (допускается опустить секунды)
setTrain	Структура TrainData (см. приложение 4) с параметрами: name - имя поезда, config - имя файла с конфигом поезда, traj - траектория, где находится голова поезда, coord - координата головы поезда вдоль траектории, dir - направление (1 - туда, -1 - обратно)	Установить поезд в заданное положение
switchFwd	"имя стрелки"	Переключить стрелку спереди
switchBwd	"имя стрелки"	Переключить стрелку сзади
delay	величина задержки, сек.	Задержка выполнения очереди команд
buildTrainRoute	"начальная траектория", "конечная траектория", направление (1 - туда, -1 - обратно)	Построить поездной маршрут. Происходит автоматический перевод стрелок и открытие всех попутных поездных сигналов
buildShuntingRoute	"начальная траектория", "конечная траектория", направление (1 - туда, -1 - обратно)	Построить маневровый маршрут. Происходит автоматический перевод стрелок и открытие всех попутных маневровых сигналов
setSwitchsAlongRoute	"начальная траектория", "конечная траектория", направление (1 - туда, -1 - обратно)	Установить стрелки по маршруту

Функция	Параметры	Описание
setTrigger	функция обработки события вида trig_func(train_name, traj_name, is_busy)	Установка триггера, срабатывающего при занятии или освобождении какой-либо траектории. Происходит вызов функции trig_func (имя может быть произвольным) с передачей туда имени поезда (train_name), имени траектории, изменившей состояние (traj_name), и нового состояния занятости (is_busy)
setTimeTrigger	"hh:mm:ss" (или "+hh:mm:ss"), имя исполняемой функции trig_func	Установка исполнения произвольной функции trig_func в заданный момент времени. Время может быть абсолютным ("hh:mm:ss") или задано относительно времени старта игры ("+hh:mm:ss"). Допускается не указывать секунды
setPostEventTimeTrigger	"hh:mm:ss", функция для исполнения trig_func	Устанавливает триггер, срабатывающий через указанный промежуток времени с момента установки данного триггера. При срабатывании устанавливается заданная функция trig_func
openShuntingSignal	"имя коннектора", направление сигнала (1 - туда, -1 - обратно)	Открыть маршрутный сигнал, установленный на коннекторе с заданным именем в заданном направлении
openCallSignal	"имя коннектора", направление сигнала (1 - туда, -1 - обратно)	Открыть пригласительный сигнал, установленный на коннекторе с заданным именем в заданном направлении

Функция	Параметры	Описание
getTrajState	"имя траектории"	Получить состояние траектории с данным именем. Возвращает два параметра (см. приложение 4): is_busy - занятость (true/false), in_route - включенность в уже построенный маршрут (true/false)
logMessage	"текст сообщения"	Вывести произвольный текст в logs/journal.log
getNextTraj	"имя траектории", направление (1 - туда, -1 - обратно)	Получить имя траектории, следующей за указанной траекторией, в заданном направлении
renameTrain	"текущее имя", "новое имя"	Переименовать поезд

Приложение 2. Перечень action-функций для подстановки в триггеры

Функция	Параметры	Описание
actionBuildTrainRoute	"начальная траектория", "конечная траектория", направление (1 - туда, -1 - обратно)	Построение поездного маршрута от начальной до конечной траектории в заданном направлении. Автоматическая установка стрелок по маршруту и открытие всех попутных поездных сигналов. Маршрут подсвечивается в карте желтой линией
actionBuildShuntingRoute	"начальная траектория", "конечная траектория", направление (1 - туда, -1 - обратно)	Построение маневрового маршрута от начальной до конечной траектории в заданном направлении. Автоматическая установка стрелок по маршруту и открытие всех попутных маневровых сигналов. Маршрут подсвечивается в карте желтой линией
actionSetSwitchsAlongRoute	"начальная траектория", "конечная траектория", направление (1 - туда, -1 - обратно)	Устанавливает стрелки по маршруту от начальной до конечной траектории в заданном направлении. Маршрут НЕ подсвечивается в карте. Попутные сигналы НЕ открываются
actionOpenShuntingSignal	"имя коннектора", направление (1 - туда, -1 - обратно)	Открытие маневрового сигнала, установленного на коннекторе с заданным именем в заданном направлении
actionOpenCallSignal	"имя коннектора", направление (1 - туда, -1 - обратно)	Открытие пригласительного сигнала, установленного на коннекторе с заданным именем в заданном направлении

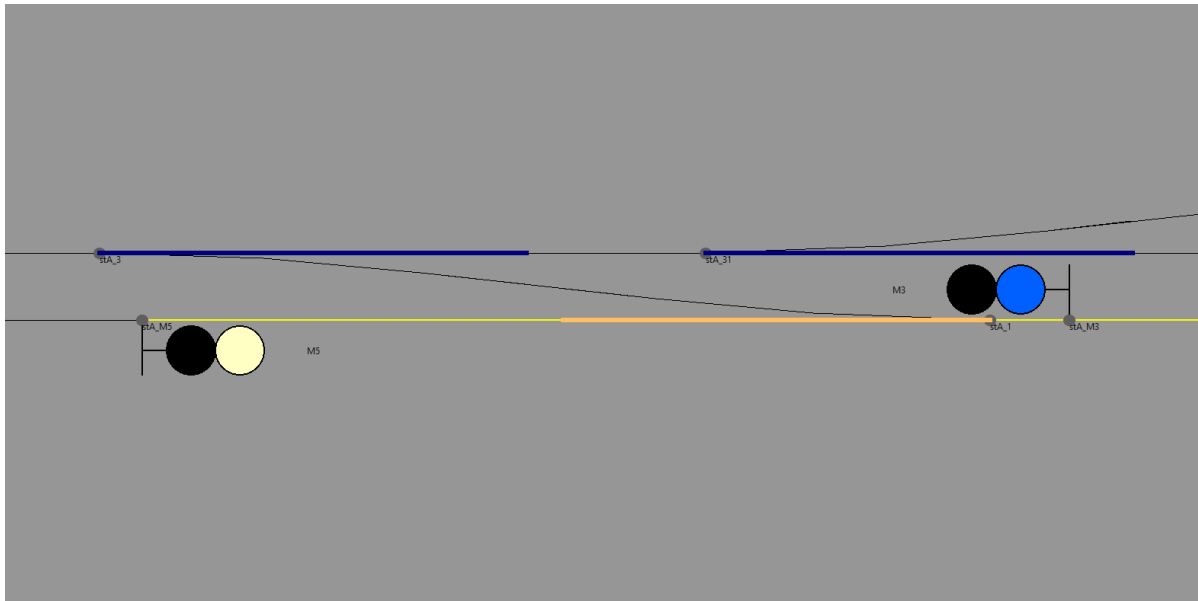


Пример 3. Открытие маневрового сигнала в заданное время

```
setTimeTrigger("+00:00:20", actionOpenShuntingSignal("stA_M5", 1))
```

Открывает маневровый сигнал M5, установленный на коннекторе с именем "stA_M5", в направлении "туда" (1), через 20 секунд после старта игры.

Результат работы в карте



ВНИМАНИЕ!!! Для отображения имен коннекторов, включите их отображение в карте "Вид -> Показать имена стрелок и стыков"

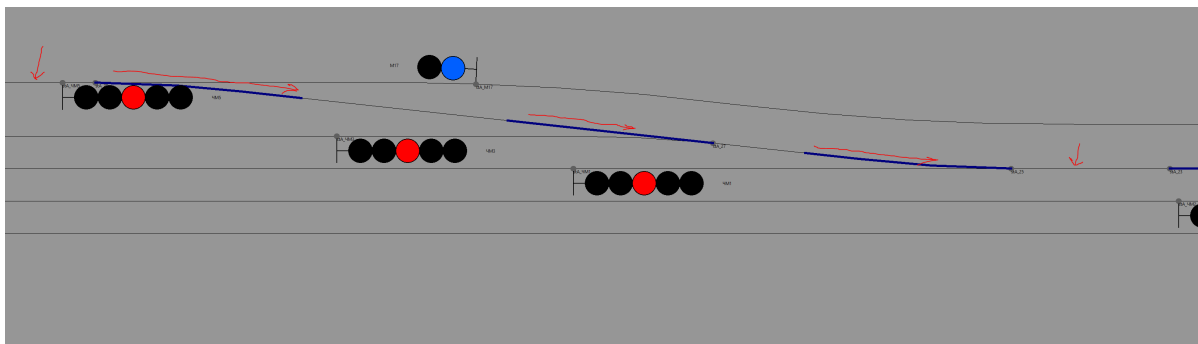
ОТДЕЛЬНОЕ ВНИМАНИЕ!!! В именах коннекторов (стрелок и стыков), а также в литерах сигналов ДОПУСКАЕТСЯ применение кириллических символов! Будьте внимательны, обратитесь за справкой к автору маршрута, в случае непонятной ситуации! В данном примере литер маневрового сигнала и имя его коннектора содержат русскую букву "М".

Пример 4. Установка стрелок по маршруту в заданное время

```
setTimeTrigger("+00:00:12", actionSetSwitchsAlongRoute("track_a_p5a",  
"track_a_25-23", 1))
```

Устанавливает по маршруту стрелки от пути 5А станции А ("track_a_p5a") до стрелочного участка между стрелками 25 и 23 ("track_a_25-23") в направлении "туда" (1).

Результат работы в карте - стрелки 29, 27 и 25 переведены в положение, обеспечивающее проезд с пути 5А на стрелочный участок 25-23.

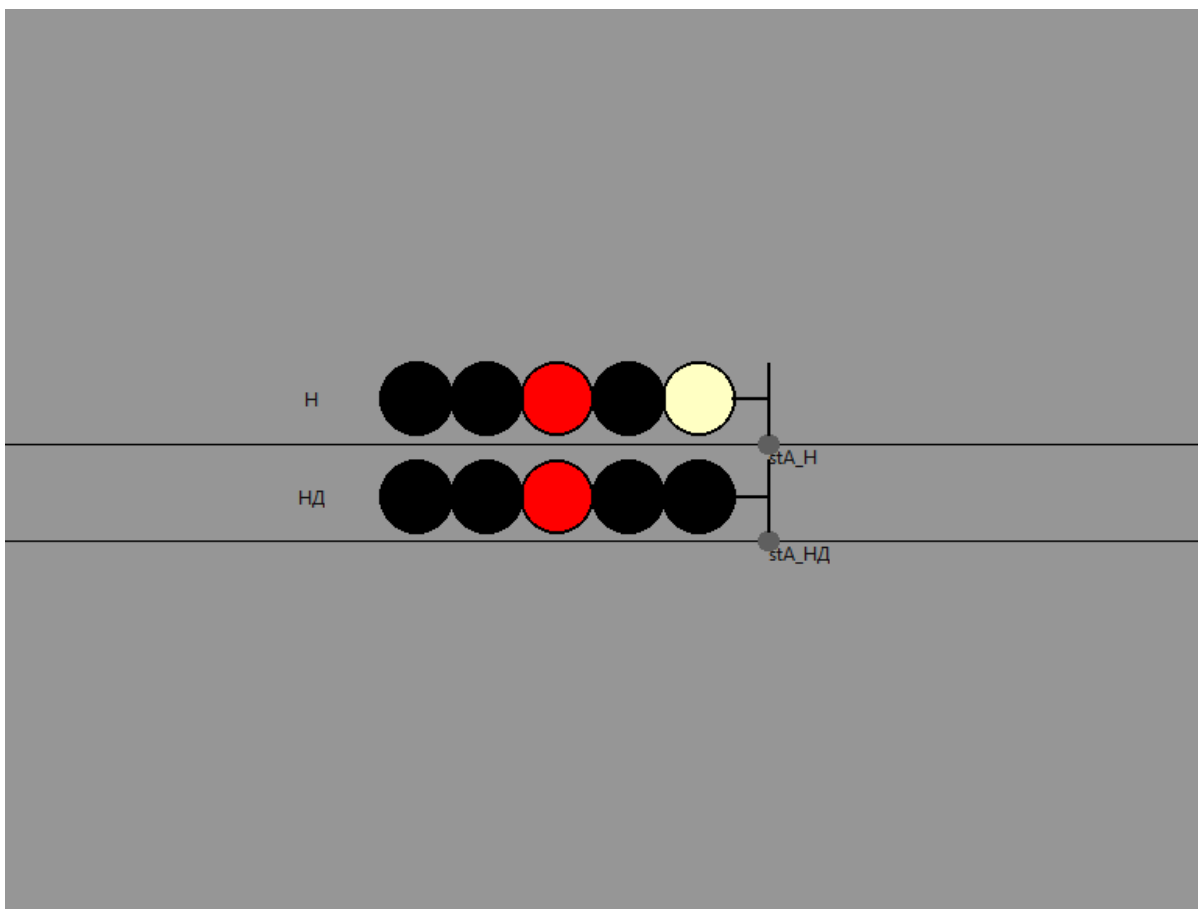


Пример 5. Открытие пригласительного сигнала в заданное время

```
setTimeTrigger("+00:00:15", actionOpenCallSignal("stA_H", -1))
```

Открывает пригласительный сигнал Н на входном светофоре станции А в направлении "обратно" (-1). Сигнал установлен на коннекторе "stA_H".

Результат работы на карте



Пример 6. Закрытие пригласительного сигнала в заданное время.

```
setTimeTrigger("+00:00:25", actionCloseSignal("stA_H", -1))
```

Закрывает пригласительный сигнал Н на входном светофоре станции А в направлении "обратно" (-1). Сигнал установлен на коннекторе "stA_H".

Пример 7 Переименование поезда в заданное время

```
setTimeTrigger("12:01", actionRenameTrain(train134.name, "136"))
```

Переименует поезд, имя которого хранится в переменной `train134.name`, присвоив ему имя "136".

Приложение 3. Стандартные позиционные триггеры

Функция	Параметры	Назначение
setOnTrajBusyTrigger	"имя траектории", выполняемая функция (action)	Выполнение заданной action-функции при занятии указанной траектории. Срабатывает однократно.
setOnTrajFreeTrigger	"имя траектории", выполняемая функция (action)	Выполнение заданной action-функции при освобождении указанной траектории. Срабатывает однократно.
setOnTrajBusyByTrainTrigger	"имя траектории", "имя поезда", выполняемая функция (action)	Выполнение заданной action-функции при занятии указанной траектории поездом с указанным именем. Срабатывает однократно.
setOnTrajFreeByTrainTrigger	"имя траектории", "имя поезда", выполняемая функция (action)	Выполнение заданной action-функции при освобождении указанной траектории поездом с указанным именем. Срабатывает однократно.
setAutoApproachTrigger	"имя начальной траектории", "имя конечной траектории", направление (1 - туда, -1 - обратно)	Автоматически строит маршрут в заданном направлении между начальной и конечной траекториями, по факту занятия начальной траектории любым поездом. Существует в течение всей игры с момента своей установки. При построении маршрута учитывается занятость возможных промежуточных траекторий и стрелок. Может использоваться для автоматического пропуска поездов по станции в заданном направлении.

Примечание: в качестве action-функции для перечисленных триггеров может быть использована ЛЮБАЯ из action-функций, описанных в [приложении 2](#), а также любая функция, созданная разработчиком сценария. В качестве action-функции НЕ МОЖЕТ быть использована системная функция (см. [приложение 1](#)), однако данное ограничение можно обойти, написав для нужной функции Lua-обертку.

Приложение 4. Стандартные структуры данных сценариев

TrainData - структура описания поезда

Имя поля	Назначение	Тип данных
name	имя поезда	строка
config	имя файла конфига поезда	строка
traj	имя траектории, на которой расположен поезд	строка
coord	координата поезда на траектории	число от 0 до длины траектории, м
dir	направление головы поезда	Целое число: 1 - к конечной точке траектории, -1 - к начальной точке траектории
auto	включение автоматического управления движением поезда	Логический: true - включено, false - выключено. Выключено по-умолчанию, можно не указывать для поездов, управляемых вручную

Пример использования

```
-- Устанавливаем поезд 102

-- Создаем структуру в памяти игры
train102 = TrainData.new()

-- Задаем параметры
train102.name = "102"
train102.config = "v160pk-1543-T65_17"
train102.traj = "track_b_p4"
train102.coord = 1250.0
train102.dir = 1
train102.auto = true

-- Устанавливаем поезд в игре
setTrain(train102)
```

TrajState - структура состояния траектории

Имя поля	Назначение	Тип данных
is_busy	занятость траектории подвижным составом	true - занята, false - свободна

Имя поля	Назначение	Тип данных
in_route	признак включения траектории в поездной или маневровый маршрут	true - траектория включена в маршрут, false - траектория не входит ни в какой маршрут

Пример использования

```
-- Запрос состояния траектории "track_a_pla"
local traj_state = getTrajState("track_a_pla")

-- Проверяем занятость траектории
if traj_state.is_busy then

    -- Действия, которые мы предпримем, если траектория занята

end

-- Проверяем включение траектории в маршрут
if traj_state.in_route then

    -- Действия, котоые мы предпримем, если траектория включена в маршрут

end
```

Приложение 5. Формат графика движения поезда

- Графики движения располагаются в каталоге timetable папки со сценарием. Этот каталог должен располагаться рядом с файлом main.lua
- График движения - это файл формата XML, имя которого должно совпадать с именем поезда, заданным в сценарии или присвоенным в процессе игры. Например, для поезда с именем "134" имя файла графика должно быть 134.xml

Пример графика ([сценарий trains_134_135](#) из каталога примеров)

```
<?xml version="1.0" encoding="UTF-8"?>
<Config>

  <Station>
    <Name>Станция A</Name>
    <DepartureTime>12:00</DepartureTime>
    <TargetTraj>track_a_p2b</TargetTraj>
    <TargetCoord>1040.0</TargetCoord>
    <RemovalTraj>track_a-b_nd-22</RemovalTraj>
  </Station>

  <Station>
    <Name>Станция B</Name>
    <ArrivalTime>12:20</ArrivalTime>
    <DepartureTime>12:25</DepartureTime>
    <TargetTraj>track_b_p4</TargetTraj>
    <TargetCoord>1250.0</TargetCoord>
    <ApproachTraj>track_a-b_4-2</ApproachTraj>
    <RemovalTraj>track_b-c_nd-22</RemovalTraj>
  </Station>

  <Station>
    <Name>Станция C</Name>
    <ArrivalTime>12:45</ArrivalTime>
    <TargetTraj>track_c_p3</TargetTraj>
    <TargetCoord>1250.0</TargetCoord>
    <ApproachTraj>track_b-c_4-2</ApproachTraj>
  </Station>

</Config>
```

Параметры описания станции (путевой точки) в графике движения

Параметр	Тип значения	Назначение
Name	Строка	Название станции
ArrivalTime	Строка hh:mm	Время прибытия
DepartureTime	Строка hh:mm	Время отправления

Параметр	Тип значения	Назначение
TargetTraj	Строка	Имя целевой траектории
TargetCoord	Число	Координата вдоль целевой траектории, м
ApproachTraj	Строка	Имя траектории, при достижении которой и от которой строится маршрут прибытия поезда (участок приближения)
RemovalTraj	Строка	Имя траектории, в направлении которой строится маршрут отправления (участок удаления)
IsRightPlatform	Логическое (1 или 0)	Наличие или отсутствие платформы справа от целевой траектории. Может не указываться, по-умолчанию платформа отсутствует
IsLeftPlatform	Логическое (1 или 0)	Наличие или отсутствие платформы слева от целевой траектории. Может не указываться, по-умолчанию платформа отсутствует
IsVisible	Логическое (1 или 0)	Отображение станции в графике движения. По-умолчанию, если данный параметры не указан, станция отображается